# Assignment 2

## COSC2767 - System Deployment and Operations

Tom Huynh

**Saigon-Team-2**

| | |
|---|---|
| Ma Thanh Long Mason | S4042629 |
| Do Hao Nhien | S3994477 |
| Tran Hoang Nguyen | S4054071 |
| Bui Linh Nhi | S4086154 |
| Nguyen Tran Nhat Huy | S3978239 |


Submission Date: Sep 16, 2026



# Table of Content


**Table of Content** **1**
**List of Figures** **3**
**List of Tables** **6**
**1. CI/CD Pipeline Solution Description** **7**
1.1 Problem Statement 7
1.2 Proposed solution 7
1.3 Features 8
1.3.1 Automation and release management 8
1.3.2 Testing and deployment safeguards 8
1.3.3 Runtime, security and data protection 8
1.3.4 Monitoring 8
1.3.5 Additional feature(s) 9
**2. Main Requirement Section** **9**
2.1 Cloud Integration 9
2.2 Containerised Services 10
2.3 Continuous Integration 10
2.4 Continuous Deployment 11
2.5 Testing Framework 12
2.6 Automation Tool 12
2.7 Configuration Management 13
2.8 Automated Alerting 14
2.9 Orchestration 14
**3. Advanced Requirement Section** **15**
3.1 Monitoring and Alerting 15
3.2 Blue-Green Deployment 16
3.3 Secrets Management 16
3.4 Automated Rollback 16
3.5 Database Backup and Restore 17
3.6 Static Analysis and Image Scanning 17
3.7 Centralised Logging 17
**4. Application Flow** **18**
4.1 Runtime application architecture 18
4.1.1 Explanation 19
4.2 CI/CD pipeline sequence 20
4.2.1 Explanation 20
**5. Known Bugs/Problems** **23**
**6. Self Evaluation and Reflection** **24**
6.1 Self-evaluation against our completed requirements 24

6.2 Reflection on industry practice from the guest lecture 24
6.3 Why we believe this is the appropriate solution 26
6.4 Failure recovery, rollback and scalability 26
6.4.1 Measured behaviour 26
6.4.2 Scalability 27
6.4.3 Challenges, and how they were addressed 27
6.5 Automation, efficiency, reliability, scalability, repeatability 28
6.6 Where the pipeline is still weak 29
**7. Project Responsibilities 30**
**8. Conclusion 30**
**References 32**
**Appendix A: Main Requirements Evidence 33**
A.1 Cloud Integration Figures 33
A.2 Containerised Services 35
A.3 Continuous Integration 37
A.4 Continuous Deployment 40
A.5 Testing Framework 43
A.5.1 Figures 43
A.5.2 Tests list 45
Backend Tests 45
Frontend Tests 47
End-to-End Tests 47
Recovery Tests 47
A.6 Automation Tool 47
A.7 Configuration Management 48
A.8 Automated Alerting 49
A.9 Orchestration 51
**Appendix B: Evidence For Advanced Requirements 52**
**Appendix C: Evidence for Section 6 56**
**Appendix D: Tools & Technologies Table 62**
**Appendix E: Contribution table 64**

# List of Figures

**Figure A1.1.** *Stacks list*

**Figure A1.2.** *VPC with public/private subnets across two AZs*

**Figure A1.3.** *Repositories in ECR*

**Figure A1.4.** *Load balancers*

**Figure A1.5.** *RDS database*

**Figure A1.6.** *Secrets on Secrets Manager*

**Figure A1.7.** *rmit-store-alerts SNS topic and subscribed email*

**Figure A2.1.** *Frontend Dockerfile*

**Figure A2.2.** *Backend Dockerfile*

**Figure A2.3.** *Backend Entryfile*

**Figure A2.4.** *Staging node docker services*

**Figure A3.1.** *Successful CI stages in the Jenkins application pipeline*

**Figure A3.2.** *GitHub push and a 5 minute SCM poll automatically triggering the Jenkins application pipeline*

**Figure A3.3.** *Checkout*

**Figure A3.4.** *Static analysis*

**Figure A3.5.** *Validate tools and templates*

**Figure A3.6.** *Build images with the immutable tag*

**Figure A4.1.** *Prepare staging inventory*

**Figure A4.2.** *Configure staging*

**Figure A4.3.** *Deploy staging*

**Figure A4.4.** *Verify staging deployment*

**Figure A4.5.** *Select inactive production colour*

**Figure A4.6.** *Successful CD stages in the Jenkins production pipeline*

**Figure A5.1.** *Backend tests*

**Figure A5.2.** *PostgreSQL, Backend Integration tests*

**Figure A5.3.** *Frontend tests 1*

**Figure A5.4.** *Frontend tests 2*

**Figure A5.5.** *End-to-end test*

**Figure A5.6.** *Restore Test*

**Figure A6.1.** *Jenkins jobs list*

**Figure A7.1.** *Ansible collections*

**Figure A7.2.** *Ansible collections install*

**Figure A8.1.** *Sending SNS email*

**Figure A8.2.** *Success Email*

**Figure A8.3.** *Failure Email*

**Figure A9.1.** *Staging Docker swarm*

**Figure A9.2.** *Production Docker swarm*

**Figure A9.3.** *Traffic switching*

**Figure B1.** *CloudFormation console showing the completed stack statuses*

**Figure B2.** *CloudFormation outputs showing production URL, target groups, and current production colour*

**Figure B3.** *CloudWatch dashboard showing target errors, latency, request count, and CPU utilisation*

**Figure B4.** *CloudWatch alarm configuration showing the 80% CPU threshold and evaluation periods*



**Figure B5.** *CloudFormations parameters showing different ports for different deployment colors*

**Figure B6.** *CloudWatch log group showing /rmit-store/production/application*

**Figure B7.** *Recent application log stream showing GET /healthz/ HTTP/1.1" 200*

**Figure C1.** *Build 12 of the production pipeline: sixteen stages green in 7 min 50 s, ending "Release 065d6c03 is live on 'green'". Per-stage timings are visible, including the database snapshot at 3 min and the traffic switch at 43s*

**Figure C2.** *The public endpoint reporting release 065d6c03 after build 12. This ties every screenshot that follows to one specific pipeline run*

**Figure C3.** *The storefront served through the production ALB, with product imagery loaded from the shared S3 bucket by presigned URL*

**Figure C4.** *Brand listing with working price and rating filters, four of four products rendering their images*

**Figure C5.** *Product detail with price, SKU, stock and four seeded customer reviews. The seeding stage produced usable data, not merely a task that exited zero*

**Figure C6.** *The alert from the rollback drill in build 9. The subject is labelled DRILL, the body opens "PLANNED ROLLBACK DRILL", and it reports "Rollback: SUCCEEDED - production is healthy on 'blue'". Automatic recovery and failure alerting are evidenced together*

**Figure C7.** *The production CloudWatch dashboard. The target 5xx widget reads "No data available" across the whole window, an independent corroboration of the curl sampling; the two CPU spikes are the two deployments*

**Figure C8.** *Six alarms, all OK. The five production alarms show "Actions enabled" and publish to SNS; the staging alarm shows "No actions", so alerting was attached deliberately rather than switched on everywhere*

**Figure D1.** *Runtime application architecture*

**Figure D2.** *CI/CD pipeline sequence*

**Figure D3.** *Temporary Docker build contexts*

**Figure D4.** *Staging runtime*

# List of Tables

**Table A.** *Tools & Technologies*

**Table B.** *Pipeline stages*

**Table C.** *Jenkinsfile descriptions*

**Table D.** *Measurement statistics*

**Table E.** *Contribution table (minimal)*

**Table F.** *Contribution table (full)*



# 1. CI/CD Pipeline Solution Description

## 1.1 Problem Statement

Tom is an experienced web developer, responsible for building and maintaining the RMIT E-commerce Store. However, Tom lacks experience with DevOps or CI/CD, thus he constantly faces numerous challenges during deployment and testing. Tom's pain points can be summarized into these points below:

- The website still requires manual deployment, which is prone to human-error, and failure at any point during the deployment can break the website
- No automated testing, thus testing is often inconsistent or skipped under time pressure
- No deployment visibility, Tom can't easily tell what has changed after deployment, and whether the deployment failed or succeeded
- The website has a major scaling problem, which makes it difficult for Tom to expand and improve his infrastructure, especially as the website grows more and more
- No rollback path, which makes every deployment risk as a faulty patch could cause live production to break with no easy to revert it
- High-risk deployment which encourages Tom to delay improvements
- Changes from multiple contributors are not integrated continuously, causing conflicts and delays
- Manual deployment, and fragile workflow consumes the attention of Tom, leading to stress and eventual burnout

This DevOps project is meant to tackle Tom's pain points and address the current issues present in the workflow of the RMIT E-commerce Store. The solution aims to develop an efficient CI/CD pipeline with integrated automated testing and deployment, while additionally improving release visibility, recovery and scalability. The solution must provide an efficient, reliable and repeatable release process, allowing Tom to focus on developing features rather than managing deployments manually.

## 1.2 Proposed solution

Our proposed solution provides an automated continuous integration and continuous deployment (CI/CD) pipeline for the RMIT E-commerce Store. The pipeline replaces manual deployment with a repeatable process for checking code, testing the application, building container images and deploying verified releases to AWS.

The core technologies used for our solution include: Jenkins, Ansible, Docker & Docker Swarm, CloudFormation, and a custom-built setup wizard. For full details of all the technologies used, please refer to Appendix D



## 1.3 Features

The pipeline provides the following features. Their actual implementation and configuration are discussed in the subsequent sections.

### 1.3.1 Automation and release management

- Github webhook to trigger builds, along with scheduled SCM polling as a fallback
- Automated Jenkins setup, job configuration, and infrastructure provisioning
- Infrastructure defined and built through the use of CloudFormation to ensure high scalability and repeatability
- Separated staging and production environments
- Self-sustaining and self-updating infrastructure
- Automated database migrations, static-file collection, and conditional database seeding

### 1.3.2 Testing and deployment safeguards

- Backend unit, service, model and API tests
- PostgreSQL integration testing
- Frontend utility and Vue component tests
- Browser-based end-to-end tests
- ShellCheck and cfn-lint static analysis
- Health, readiness and release-version checks before production promotion
- Blue-green deployment with inactive-colour verification before public traffic switching
- Automatic traffic rollback following failed post-switch verification, plus a deliberate rollback-drill option

### 1.3.3 Runtime, security and data protection

- Multinode Docker Swarm orchestration with application replicas, service health checks and rolling-update safeguards
- Load-balanced application access through an Application Load Balancer (ALB)
- Shared PostgreSQL storage in private AWS RDS and shared product and image storage in AWS S3
- Encryption and versioning for media buckets
- Secrets management using AWS Secrets Manager, SSM Parameter Store and Jenkins credentials, with Docker Swarm secrets for production runtime credentials
- Premigration RDS snapshots and automated database backups
- An isolated database-restore rehearsal with data verification, guarded cleanup and archived recovery reports

### 1.3.4 Monitoring

- CloudWatch dashboards and alarms for CPU utilisation
- Centralised staging and production container logs with defined retention periods



- Email notifications for deployment outcomes and configured alarm state changes
- Jenkins build history and a consolidated pipeline-status view

### 1.3.5 Additional feature(s)

- A custom setup wizard providing prerequisite checks, environment setup, credential retrieval, Jenkins access, restore rehearsal and full reset script. Full details on the wizard can be viewed in the “Manual” section in the README file.

# 2. Main Requirement Section

*For a full list of main requirement figures, refer to Appendix A.*

## 2.1 Cloud Integration

Our infrastructure is deployed on AWS using CloudFormation, three application-infrastructure stacks, plus the separate Jenkins bootstrap stack, so in total four, as shown in Figure A1.1. These stacks are linked through cross-stack exports. The four stacks are:

- **jenkins-bootstrap.yaml:** provisions the Jenkins EC2 server in a selected VPC and subnet, with an Elastic IP, security group and a Secrets Manager secret for the Jenkins administrator password. It installs and initialises Jenkins and the required deployment tools, providing the automation server that runs the infrastructure, application, production and restore-rehearsal jobs.

- **shared-services.yaml:** a shared VPC with two public subnets and two private DB subnets across two AZs, an Internet Gateway with public routing, two ECR repositories (rmit-store/backend, rmit-store/frontend), as shown in Figures A1.2-A1.3; and an SNS topic (rmit-store-alerts) used for email alerting.

- **staging.yaml:** a single EC2 node (Swarm single-node cluster) behind an Application Load Balancer, a security group restricted to the ALB and a configurable admin CIDR, an S3 bucket for staging media, Secrets Manager secrets for the Django secret key and DB credentials (Figure A1.6), a CloudWatch log group, and a CPU CloudWatch alarm.

- **production.yaml:** a multi-node EC2 Swarm cluster (manager + workers) spread across two AZs, and RDS PostgreSQL instance, an S3 bucket for production media, and an ALB in front of the Swarm ingress. See Figures A1.4-A1.5 for graphics.

**Design decision:** splitting AWS resources into four stacks rather than simply using one for everything means the network and container registries are provisioned once and reused, while the two environments (staging, production) can be updated, taken down, or recreated independently without affecting each other. This keeps blue/green production cut-over and staging resets safe.



We also used AWS ECR for image registry, ALB for traffic routing/health checks, Secrets Manager for credential storage (referenced at deploy time, not baked into images), CloudWatch for logging + alarms via the awslogs Docker logging driver, SNS for alerting (Figure A1.7), and RDS for the database.

## 2.2 Containerised Services

For the frontend, we used a multi-stage Docker build (***docker/client/Dockerfile***) (Figure A2.1). Node 24 Alpine builds the Vue bundle, then an Nginx Alpine image serves the static files and reverse proxies API/static calls to the backend service.

For the backend (***docker/server/Dockerfile***) (Figure A2.2), we used a Python 3.14-slim image, dependencies installed from requirements.txt, runs as a non-root user (appuser), served by Gunicorn (with 3 workers) via start.sh (Figure A2.3).

At the database tier, it is containerised in staging and managed in RDS in production. Staging runs postgres:16-alpine as a Swarm service with a named local volume, isolated on an internal-only db-net overlay network (Figure A2.4). This keeps the staging environment cheaper, fast to reset and self-contained for CI verification. Production uses AWS RDS for PostgreSQL instead of a container because RDS gives automated backups, storage encryption, and independent lifecycle/patching that a container on the same host as the application cannot provide; it also means a backend container failing or being rescheduled by Swarm never risks the data volume, and the database can survive redeploys of the application stack entirely.

Because production runs BACKEND_REPLICAS > 1 spread across AZs, any file written to a container's local filesystem would only be visible to that one replica and would be lost on rescheduling. To solve this, media storage is externalised to S3 rather than a container volume.

## 2.3 Continuous Integration

CI is implemented through ***jenkins/Jenkinsfile.application*** (Figure A3.1), triggered automatically on every push via ***triggers { githubPush(); pollSCM('H/5 * * * *') }*** (GitHub webhook, with a 5 minute SCM poll as fallback) as shown in Figures A3.1 and A3.2. Relevant pipeline stages include:

- **Checkout:** Checks out the configured branch of the assigned private repository, sets RELEASE_VERSION to its full commit SHA, and generates a unique IMAGE_TAG (Figure A3.3).
- **Static analysis:** runs scripts/static-analysis.sh (Figure A3.4).
- **Backend tests** (Figure A5.1)**:** runs pytest inside a disposable python:3.14-slim-bookworm container against the backend source.
- **Backend PostgreSQL integration tests** (Figure A5.2)**:** runs scripts/test-backend-postgres.sh to run tests against a real Postgres instance rather than mocks.
- **Frontend tests** (Figure A5.3-A5.4)**:** runs vitest inside a node:24-alpine container.



- **Validate tools and templates:** validates all 3 CloudFormation templates with aws cloudformation validate-template (Figure A3.5).
- **Build images:** builds backend and frontend Docker images tagged with the immutable IMAGE_TAG (Figure A3.6).

Every stage's pass/fail is reported back through the Jenkins build result (and console log), and a failure at any point halts the pipeline and prevents subsequent stages from running.

**Design decision:** tests run inside throw-away Docker containers (not on the Jenkins agent directly) so the CI environment exactly matches the language/runtime versions used in the production images, so the Jenkins host itself needs no website-specific tooling installed.

## 2.4 Continuous Deployment

The same ***Jenkinsfile.application*** pipeline continues past CI into CD, deploying to staging first and only promoting to production if staging passes:

- **Prepare staging inventory** (Figure A4.1)**:** looks up the staging EC2 instance/ALB DNS/media bucket from CloudFormation outputs and generates an Ansible inventory.
- **Configure staging/Deploy staging** (Figure A4.2-A4.3)**:** runs Ansible playbooks (configure-staging.yml, deploy-staging.yml) to push the new images and render/deploy the Swarm stack
- **Verify staging deployment** (Figure A4.4)**:** hits /healthz/, /readyz/, /api/version/ (checked against the new RELEASE_VERSION), /api/products/, and / with retries, failing the build if staging isn't actually healthy.
- **Staging end-to-end tests** (Figure A5.5)**:** runs Playwright end-to-end tests against the live staging URL.
- **Select inactive production color** (Figure A4.5)**/Promote to production:** only after every prior stage succeeds, the pipeline determines the currently inactive blue/green production color and triggers the downstream rmit-store-production job to deploy there.

***jenkins/Jenkinsfile.production*** (Figure A4.6) then deploys the new version to the currently idle color, runs database migrations (***migrate-production.yml***), verifies the new color is healthy, then switches ALB/router traffic (***scripts/switch-production-color.sh***) to the new color. If live verification fails after the switch, the pipeline automatically switches back to the previous color and reports rollback status (***SUCCEEDED***/***PARTIAL***/***FAILED***).

For failure handling, if any test/build/deploy stage fails, the ***post { failure { … } }*** block calls ***notifyTeam(...)***, which publishes to the shared SNS alert topic, and the pipeline stops. Production is only updated by the explicit "Promote to production" stage, which is unreachable unless every previous stage succeeded.

**Design decision:** promoting to production is a separate downstream Jenkins job so that blue/green color selection, DB migration, and rollback logic are isolated and reusable, and so production deploys can be triggered/audited independently of the app pipeline if ever needed.



## 2.5 Testing Framework

The project uses several testing frameworks because each layer requires a different testing approach:

- **Backend:** pytest and pytest-django are used for unit, database, service, and Django API tests. They provide fixtures, database isolation, parametrized tests, and clear assertions.
- **Backend API:** Django REST Framework's API Client tests authenticated and unauthenticated HTTP requests without needing a browser.
- **Concurrency:** PostgreSQL is used for the checkout concurrency test because real row-lock behavior cannot be reliably tested with SQLite.
- **Frontend:** Vitest with jsdom tests JavaScript utilities in a browser-like environment.
- **Vue components:** @vue/test-utils mounts Vue components and tests props, interactions, and emitted events.
- **End-to-end:** Playwright tests the deployed staging application through a real browser.
- **Recovery:** Python's built-in unittest and unittest.mock test recovery logic with simulated AWS responses, avoiding changes to real infrastructure.
- **Database recovery verification:** A standalone Python script checks the restored database and verifies the application through HTTP.

For test figures, refer to Appendix A.5.1. For a list of all tests and their purposes, refer to Appendix A.5.2.

## 2.6 Automation Tool

Jenkins orchestrates the entire pipeline end-to-end, matching the required stages directly to Jenkinsfile stages:

| Required stage | Implementation |
|---|---|
| Buid frontend bundle | docker build on docker/client/Dockerfile (multi-stage npm run build) |
| Install backend dependencies | pip install -r requirements.txt inside the backend Docker build and inside the test container |
| Run migrations | ansible/playbooks/migrate-production.yml, run before traffic switch |
| Run test suites | Backend pytest, backend Postgres integration tests, frontend vitest, staging Playwright E2E |
| Publish results | Jenkins Stage View / console log per stage; SNS notification on failure |
| Build and push container images | "Build images" + "Push images to ECR" stages, tagged with a unique, immutable IMAGE_TAG |



| | |
|---|---|
| Deploy | Ansible-driven docker stack deploy to staging, then blue/green Swarm deploy to production |

**Table B**. *Pipeline stages*

**Four Jenkinsfiles separate concerns:**

- **Jenkinsfile.infrastructure:** provisions/updates CloudFormation stacks
- **Jenkinsfile.application:** CI + staging CD, described above
- **Jenkinsfile.production:** blue/green production promotion
- **Jenkinsfile.restore-test (Figure A5.6):** an on-demand, parameterised database-restore check, launched through Jenkins or the setup wizard

**Design decision:** infrastructure provisioning (Figure A6.1) is separated from application deployment to keep resource management and release logic independently maintainable. The application pipeline invokes the rmit-store-foundation job to check and deploy the CloudFormation stacks before proceeding. CloudFormation deployment commands run on each invocation, but unchanged stacks require no resource updates.

## 2.7 Configuration Management

Ansible manages all server-side configuration, organised into roles under ansible/roles/ (Figure A7.1):

- **docker:** installs and configures the Docker Engine and Swarm prerequisites on all nodes.
- **swarm-manager/swarm-worker:** initialises the Docker Swarm cluster (manager on the staging node and production manager; workers join production for horizontal scale).
- **staging-application:** renders the Jinja2 swarm/staging-stack.yml template with environment values pulled from ansible/group_vars/staging.yml and secrets fetched from AWS Secrets Manager at deploy time, then runs docker stack deploy.
- **production-application:** equivalent role for production, feeding swarm/production-stack.yml.

Playbooks (***configure-staging.yml, deploy-staging.yml, configure-production.yml, deploy-production.yml, migrate-production.yml, seed-production.yml***) are invoked directly from the Jenkins pipeline over SSH (***sshagent(credentials: ['aws-ec2-key']***), using dynamically generated inventories built from CloudFormation stack outputs. (Figure A7.2)

**Design decision:** Secrets are retrieved from AWS Secrets Manager during deployment rather than committed to the repository or baked into application images. Staging supplies them through a temporary rendered stack file and container environment variables, the rendered file is removed after deployment. Production delivers runtime credentials through Docker Swarm secrets.



## 2.8 Automated Alerting

Email alerting is implemented via AWS SNS: the ***AlertTopic*** (with an email subscription to the team's address, provisioned conditionally on ***NotificationEmail*** in ***shared-services.yml***) is looked up by ARN from CloudFormation exports and published to by the notifyTeam() helper function defined in both ***Jenkinsfile.application*** and ***Jenkinsfile.production***.

**Alerts will fire on:**

- Build/test failure via post { failure { notifyTeam(...) } } in the application pipeline (any CI stage failing) (Figure A8.3).
- Deployment failure via post { failure { … } } in the production pipeline.
- Failed post-switch verification/rollback outcome: a dedicated notification reporting whether the automated rollback SUCCEEDED, was PARTIAL, or FAILED, so the team knows immediately if manual intervention is required.
- Production build is successful (Figure A8.2).

Each notification includes the job name, build number, branch, and build URL so the team can go straight to the failing log.

**Design decision:** alerting goes through SNS rather than Jenkins' build-in mail step (Figure A8.1) so that the same alert topic can be reused by non-Jenkins sources and so the subscription is managed centrally in infrastructure-as-code rather than in Jenkins job configuration.

## 2.9 Orchestration

Docker Swarm is used for orchestration in both environments:

- **Staging:** single-node Swarm (Figure A9.1) (the node is both manager and worker), running frontend, backend, and db services. Two overlay networks: staging-net (frontend ↔ backend, ALB-facing) and an internal db-net (backend ↔ db only; internal: true, no route out of the Swarm, no published ports), so the database is unreachable from outside the cluster even indirectly.

- **Production:** multi-node Swarm cluster (Figure A9.2) (dedicated manager + workers across two AZs). backend and frontend services are declared with replicas: ${BACKEND_REPLICAS} / ${FRONTEND_REPLICAS} and placement.preferences: spread: node.labels.availability_zone, so Swarm actively spreads replicas across AZs for availability. Rolling updates use update_config: { order: start-first, failure_action: rollback }. New tasks are started and health-checked before old ones are stopped, and Swarm itself rolls back a service update automatically if the new tasks fail their healthcheck. Logging for every service uses the awslogs driver, sending container logs straight to CloudWatch.

Combined with blue/green color switch (Figure A9.3) at the Jenkins/ALB level, this gives two layers of resilience: Swarm handles container-level rolling updates and self-healing within a



color, while the Jenkins pipeline handles environment-level blue/green promotion and rollback between colors.

**Design decision:** Swarm was chosen over Kubernetes because the required orchestration features (multi-replica services, rolling/self-healing updates, overlay networking, placement constraints) are all natively available with a much smaller operational footprint; no separate control-plane infrastructure to provision, which suited the scope and EC2 instance limit constraints of the assignment.

# 3. Advanced Requirement Section

The project uses Infrastructure as Code to create repeatable AWS environments and reduce manual configuration errors. Infrastructure is defined in four CloudFormation templates: Jenkins bootstrap, shared services, staging, and production. These templates define networking, EC2 instances, RDS, ECR repositories, load balancers, security groups, secrets, monitoring resources, and storage.

The infrastructure pipeline deploys the shared services stack before the staging and production stacks. Outputs and imports pass shared resource identifiers between stacks, avoiding repeated manual copying of VPC, subnet, repository, and notification-topic values. Environment-specific settings, such as instance sizes, key-pair names, and the active production colour, are provided through parameters.

The production database is private and is accessed through security-group rules. The production media bucket uses encryption and versioning. The templates reuse the AWS Academy-provided LabInstanceProfile; therefore, the project does not claim to provide a completely custom least-privilege IAM design, as shown in Figure B1 and B2.

## 3.1 Monitoring and Alerting

Monitoring uses Amazon CloudWatch for infrastructure and load-balancer metrics, CloudWatch Logs for application output, and Amazon SNS for notifications. The production environment contains CPU alarms for the manager and worker nodes. Each alarm uses an 80% CPU threshold, a five-minute period, and two consecutive evaluation periods. Missing data is treated as notBreaching so that periods without traffic do not automatically generate false alarms, as shown in figure B3 and B4.

The production load balancer also monitors target five-hundred errors and average target response time. The dashboard groups target errors, response time, request count, and production-node CPU utilisation in one view. Alarm and recovery notifications are sent to the shared SNS topic.

These settings detect sustained resource or application-facing problems rather than reacting to a single short spike. However, the thresholds are initial project settings and have not been



proven optimal through load testing. The metrics measure ALB targets and infrastructure; they are not a custom Django application metrics endpoint.

## 3.2 Blue-Green Deployment

The production deployment uses a blue-green strategy. Blue and green application stacks run on separate ports, 8081 and 8082, shown on figure B5. The public listener uses port 80, while Jenkins accesses colour-specific test listeners on ports 9001 and 9002.

Before switching public traffic, the pipeline identifies the inactive colour, creates a database snapshot, deploys the new release, and checks the inactive colour's health, readiness, and expected website version. The version endpoint is compared with the website commit SHA. Only after these checks pass does the pipeline change the public listener to the new colour.

This design allows the new release to be inspected before users are routed to it while retaining the previous colour as a rollback target. It is blue-green deployment rather than gradual canary deployment. Both colours share the production database, so this design does not guarantee compatibility with destructive or incompatible schema changes.

## 3.3 Secrets Management

Sensitive values are stored and delivered through several mechanisms according to their use. Database and application secrets are stored in AWS Secrets Manager. The production seed administrator credential is stored as an SSM Parameter Store SecureString and exposed to Jenkins through a credential. GitHub credentials and SSH keys are stored as Jenkins credentials. Runtime application secrets are delivered to Docker containers through Docker Swarm secrets.

This separation prevents sensitive values from being committed to application or infrastructure repositories. Ansible tasks that process secrets use no_log to reduce accidental disclosure. However, credentials may still exist temporarily in protected files, environment variables, Docker login configuration, or Jenkins runtime storage.

## 3.4 Automated Rollback

The production pipeline records the previously active colour before switching traffic. After the switch, it verifies the public endpoint. If live verification fails, the pipeline attempts to move traffic back to the previous colour and verifies that the previous colour is healthy. The result is reported as successful recovery, partial recovery, or failed recovery.

The pipeline also includes a deliberate ROLLBACK_DRILL option. This allows the team to inject a post-switch verification failure and exercise the rollback path. Docker Swarm separately uses start-first updates and failure_action: rollback for service-level update failures.



## 3.5 Database Backup and Restore

Before production migrations, the pipeline creates a named RDS snapshot and waits for snapshot completion. RDS automated backups are configured with a three-day retention period. CloudFormation deletion and replacement policies also request snapshots.

The restore process creates a temporary database from a selected snapshot rather than overwriting the live production database. The recovery verifier checks database connectivity, migration history, schema compatibility, product data, user and product counts, order totals, and access to the restored catalogue through the application API. Guarded cleanup is attempted after verification and the recovery report is archived.

## 3.6 Static Analysis and Image Scanning

The pipeline performs static analysis before infrastructure deployment. Shell scripts are checked with ShellCheck and CloudFormation templates are checked with cfn-lint. A non-zero result prevents later deployment stages. The ECR repositories also have scan-on-push enabled.

These checks catch script and template errors early and provide basic registry image scanning. However, the pipeline does not currently inspect vulnerability findings and block deployment based on severity. Therefore, the project should describe this as static analysis plus configured registry scanning, rather than claiming that all dependencies and images are security-approved.

## 3.7 Centralised Logging

Staging and production Docker Swarm services use the AWS Logs driver, shown in Figures B6 and B7. Production logs are sent to /rmit-store/production/application, while staging logs are sent to /rmit-store/staging/application. The CloudWatch log groups use seven-day retention.

Centralised logging allows the team to inspect application output without connecting individually to every EC2 node. The retention period limits unnecessary storage in the lab environment. The logging configuration forwards container console output but does not guarantee structured JSON logs, distributed tracing, or the absence of secrets in application messages.

# 4. Application Flow

## 4.1 Runtime application architecture

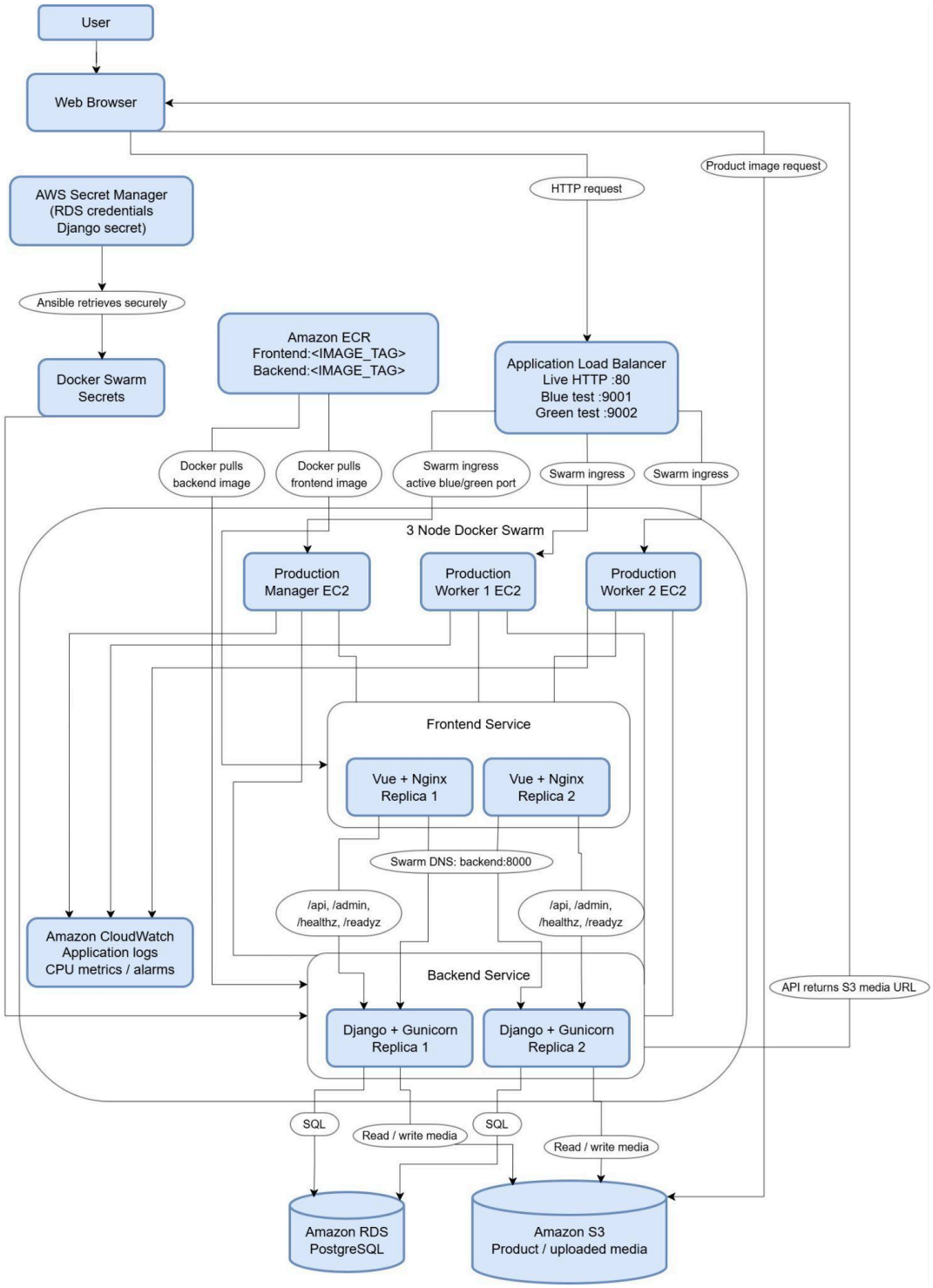



**Figure D1.** *Runtime application architecture*



### 4.1.1 Explanation

The production environment consists of three EC2 instances forming a single Docker Swarm cluster: one manager and two workers. Docker Swarm is responsible for scheduling and distributing the application containers across these nodes. The final production configuration runs two frontend replicas and two backend replicas, rather than dedicating one EC2 instance to each application component.

The frontend and backend are separate services. The frontend is a Vue application served by Nginx. Nginx is the externally reachable application service to the Docker Swarm backend service.

The Django backend is served by Gunicorn and is responsible for the application's business logic and API. Persistent data is intentionally kept outside the containers:

- Amazon RDS provides the shared PostgreSQL database, allowing every backend replica to access the same application data.
- Amazon S3 provides shared media storage so product images are not tied to the filesystem of a particular container or EC2 worker.

➔ This is necessary in a multi-node deployment because local container storage would not be shared between replicas.

Sensitive configuration is not stored in the Docker image. RDS credentials and the Django secret originate from AWS Secrets Manager. In the implemented production workflow, Ansible securely retrieves these values and converts them into Docker Swarm secrets.

Amazon ECR stores the frontend and backend application images. The current pipeline distinguishes between the logical release identifier and the Docker image identifier:

- **RELEASE_VERSION** is the full commit SHA of the assigned private repository and represents the complete release state.
- **IMAGE_TAG** is the Docker image tag generated by the pipeline for the frontend and backend images. It is derived from release/build information, including the release SHA, shortened repository SHA and timestamp.

Therefore, the application version can be traced back to the assigned repository commit through **RELEASE_VERSION**, while Docker Swarm deploys the specific container artifacts identified by **IMAGE_TAG**.

CloudWatch collects application logs from the frontend and backend services and provides infrastructure monitoring and alarms.

# 4.2 CI/CD pipeline sequence

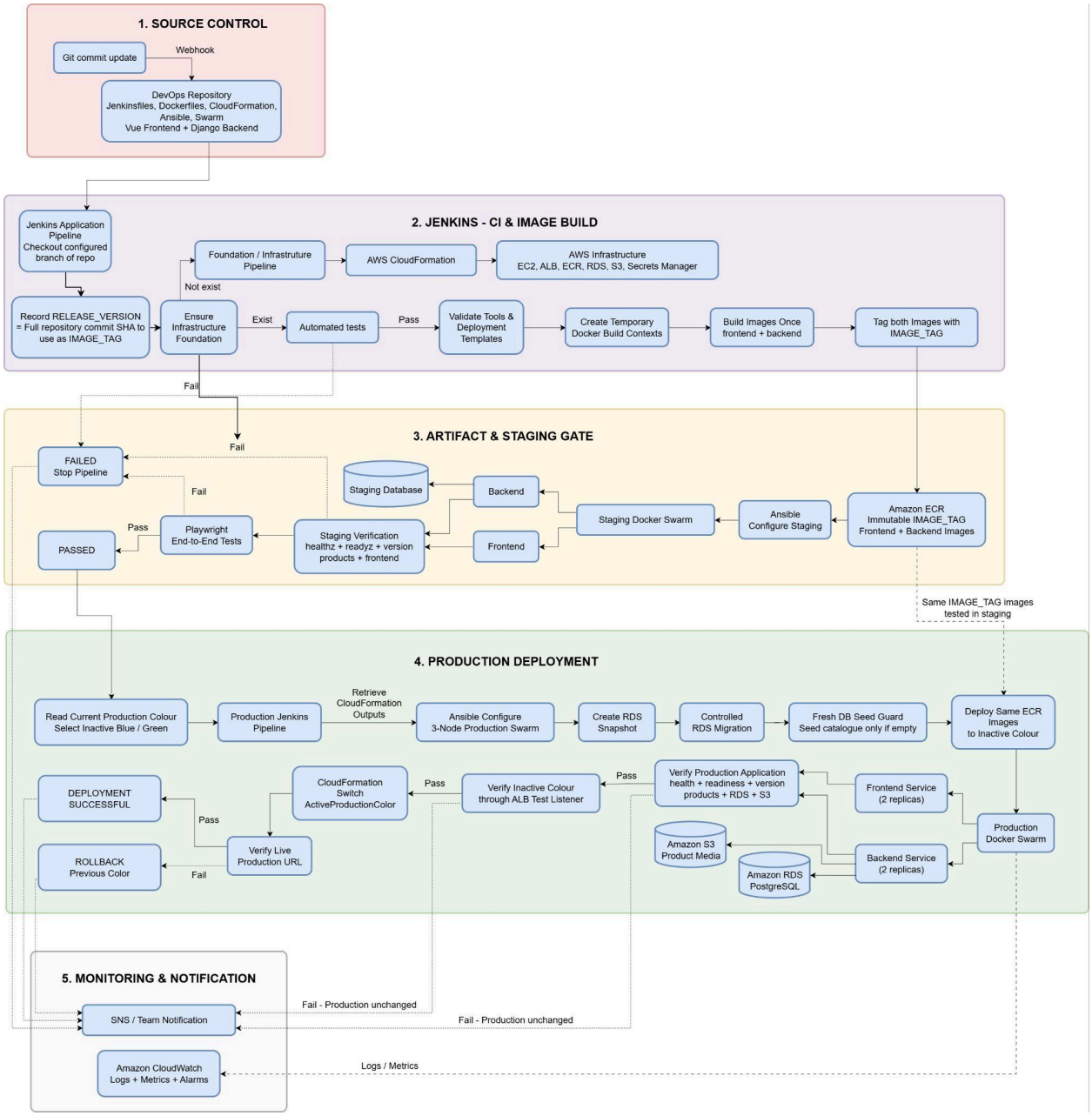



**Figure D2.** *CI/CD pipeline sequence*

## 4.2.1 Explanation

The CI/CD workflow uses only our private DevOps GitHub repository as the source of truth for the release. The repository contains:



- the imported application source under **website/**
- automated tests under **tests/**
- Jenkins pipeline definitions
- CloudFormation templates
- Ansible playbooks and roles
- Docker and Docker Swarm deployment configuration

A GitHub update triggers the Jenkins application pipeline. Jenkins checks out the configured branch of the assigned repository and records the repository's full commit SHA as **RELEASE_VERSION**. Since for this assignment, application source and deployment automation are stored in the same repository.

Before application deployment proceeds, Jenkins ensures that the infrastructure pipeline has successfully provisioned the required AWS resources. CloudFormation generates (first run where infrastructure not generated yet) and manages infrastructure including:

- EC2 instances
- Application Load Balancers
- Security groups
- ECR repositories
- RDS
- S3
- Secrets Manager
- CloudWatch

Before deployment activity continues, Jenkins performs static analysis and automated testing. The validation stage includes:

- Backend tests
- Frontend tests
- PostgreSQL integration tests
- Static analysis and other code-quality checks

After the tests and infrastructure checks pass, Jenkins creates temporary Docker build contexts by combining the application source under **website/** with the Dockerfiles under **docker/**.

```
.release-context/
├── backend/
│   ├── Django source
│   ├── Dockerfile
│   └── start.sh
│
└── frontend/
    ├── Vue source
    ├── Dockerfile
    └── nginx.conf
```

**Figure D3.** *Temporary Docker build contexts*

These temporary files exist only in the Jenkins workspace during the Docker images build to avoid modifying the imported application source.

Jenkins then produces a separate **IMAGE_TAG** for the Docker images:

- rmit-store/backend:<IMAGE_TAG>
- rmit-store/frontend:<IMAGE_TAG>

The created images are then pushed to Amazon ECR. These images are built once during the staging phase and will be pulled from ECR for deployment during the production phase.

Next, Ansible configures the staging EC2 instance as a small single-node Docker Swarm. And the staging runtime contains:

```
Staging ALB
     ↓
Frontend container
     ↓
Backend container
     ↓
PostgreSQL staging container

Backend → Staging S3
```

**Figure D4.** *Staging runtime*

The exact SHA-tagged frontend and backend images from ECR are deployed to staging. Jenkins then checks:

- /healthz/
- /readyz/
- /api/version/
- /api/products/

If staging fails, the pipeline stops and production remains unchanged.



If frontend response, then executes the Playwright end-to-end tests. After staging passes, Jenkins queries CloudFormation to find the colour currently receiving production traffic. The application pipeline then invokes the production pipeline and supplies:

- **RELEASE_VERSION:** The full commit SHA of the assigned private repository, used to verify the deployed release through /api/version/
- **IMAGE_TAG:** The generated image tag identifying the exact backend and frontend images that passed staging
- **DEPLOYMENT_COLOR:** The currently inactive production colour, either blue or green

➔ Pull the images with the same <IMAGE_TAG> tag to promote for production.

The production job dynamically retrieves CloudFormation outputs and generates the Ansible production inventory. Ansible then establishes or verifies a 3-node Docker Swarm:

- Production Manager
- Production Worker 1
- Production Worker 2

Next, for a completely new production database, the seed job checks whether products already exist to protect an existing database from being unnecessarily reseeded. The exact same ECR images that passed staging are deployed to the currently inactive production colour. The production environment supports:

- Blue → port 8081
- Green → port 8082

The inactive colour is first tested through its dedicated ALB test listener before public traffic is changed. Only after that verification succeeds does the pipeline change the CloudFormation ActiveProductionColor.

After switching traffic to the new color, Jenkins verifies the public production endpoint again. If verification succeeds, deployment is successful and the website is ready. If it fails, restore to the previous production color. This protects production from remaining on a failed release.

# 5. Known Bugs/Problems

**Jenkins tunnel connectivity problem:** The SSM port-forwarding tunnel sometimes fails to connect successfully or connects successfully but the Jenkins interface remains inaccessible from the local browser. There are several potential-causes but no concrete source has been found. If encountered, some potential fixes include: changing port used for Jenkins tunnel; checking connectivity within local OS; restarting Jenkins instance.

**Deployment branch-switch issue:** The pipeline doesn't fully support the mid-deployment branch switching. Switching the deployment branch of the pipeline may break several automated aspects of the pipeline and may require manual troubleshooting to fully repair. In



severe cases, a full reset may be required to mend the broken structure. Some potential symptoms of this issue include: Github webhook failing; Jenkins job failure.

**Secrets Manager desync:** The pipeline doesn't automate the syncing of credentials to Secrets Manager after they are changed. For example, changing the default auto-generated Jenkins password doesn't automatically update the password value in Secrets Manager, this causes the Secrets Manager to be outdated and requires manual intervention.

**Missing build URL in Emails:** Due to time limitations, the build URL in all notification emails are currently misconfigured/missing, which causes the field to appear with the value "null".

# 6. Self Evaluation and Reflection

## 6.1 Self-evaluation against our completed requirements

Our solution is four Jenkins declarative pipelines over one AWS account, each with a single responsibility:

| Pipeline | Stages | Responsibility |
|---|---|---|
| Jenkinsfile.infrastructure | 7 | CloudFormation for shared services, staging and production |
| Jenkinsfile.application | 19 | Static analysis, tests, image build, staging deploy, promotion |
| Jenkinsfile.production | 16 | Migration, seeding, blue-green deployment, cutover, rollback |
| Jenkinsfile.restore-test | 3 | Database restore rehearsal against a real snapshot |

**Table C.** *Jenkinsfile descriptions*

The application pipeline is the spine: static analysis, 18 backend `pytest` modules and 2 frontend `vitest` specs, container images pushed to ECR under a unique IMAGE_TAG, followed by staging deployment of those exact images, 2 Playwright end-to-end specs against staging, and only then a read of the live production colour to trigger the production pipeline against the idle one. No stage trusts a previous stage's word for it: each re-derives what it needs and verifies the result. Appendix C carries the evidence for this section. Figures C1 and C2 tie one pipeline run to the release the public endpoint then reported, and Figures C3 to C5 show the storefront it served.

Judged against what we set out to build, the pipeline is complete for the core path: Once the environment and credentials are configured, a push to the configured branch triggers the pipeline, which proceeds to production automatically when all required checks pass. No manual approval gate is configured. Where we fall short is described in 6.6.

## 6.2 Reflection on industry practice from the guest lecture

The guest lecture by Nguyễn Hùng Anh of Netcompany [1] was, in hindsight, a description of problems we then met ourselves. Four of his points map onto specific decisions in our repository.

**"You have to measure it."** Asked how one knows an architecture is successful, he said: *"you cannot simply say my change will make production 100 times better. You have to measure it [...] you cannot always use one tool or one measurement on the system, you have to be creative with it, for example payload, response time."* This changed how we evidence our own claim. Rather than asserting that blue-green gives zero-downtime deployment, we sampled the public endpoint every six seconds through both a cutover and a rollback and reported the actual counts (6.4). The measurement is what turned "we implemented rollback" into "rollback returned service to the previous release in 43 seconds with no failed request".

**"My spirit with CI/CD is always fail."** Asked about building CI/CD in practice he answered: *"The first thing is always fail. And the next thing will fail [...] you can see the possible outcome, but you cannot see all the possible outcomes."* Our record matches that exactly and we would rather report it than hide it. Our first successful production run came only after four consecutive failures: an unregistered pipeline parameter, a missing SSH credential, a Python interpreter that could not see the Docker SDK, and seed credentials that never reached the container. A fifth appeared later, after the branches were merged, when a colour was redeployed for the second time and the replica gate proved to have never waited at all. None of those were foreseeable from the design. Each was found only by running the pipeline against real infrastructure, which is the iterative process he described rather than a failure of planning.

**"The errors cannot be made by one person."** His strongest point for a student team was about accountability: *"The smallest piece is the PR and that is always the reviewer and the proofer. So to have any change in the system, it should be responsible for at least two people."* This is the practice we most clearly did *not* follow, and it cost us. We integrated by merging branches directly, without review, and all three defects in 6.4 reached the integration branch that way. Had a review gate existed, the interpreter mismatch and the replica count that never waited would both have been visible in the diff.

**"Architecture IS Operations."** He contrasted architecture as it is taught, static diagrams that stop at deployment, with enterprise reality, where reliability depends on naming conventions, automated gates and clear team boundaries. Our cutover script is a direct instance. The obvious way to move traffic between colours is `aws elbv2 modify-listener`, one API call. We deliberately did not do that. The script updates the `ActiveProductionColor` parameter of the production CloudFormation stack instead, which is slower, because `CurrentProductionColor` is a *stack output* that other people's code reads. That decision was vindicated without us planning it: the application pipeline, written by another team member, now queries exactly that output to decide which colour is idle. Had we taken the faster path, the output would have drifted from reality and a different pipeline would have deployed over the live colour.

Three further points landed after the fact. His OAuth case study argued that naming conventions are security gates, not administrative overhead; every Swarm secret and config we create



carries its environment and release in the name, but nothing rejects a resource that ignores the convention, which is the gap between a standard and a gate. And his dynamic-IP allowlist demonstration named a failure we then hit: our per-colour test listeners on ports 9001 and 9002 are restricted to the Jenkins CIDR, and the lab reassigns the Jenkins public IP on every session restart, so the rule silently stopped matching until we attached an Elastic IP.

## 6.3 Why we believe this is the appropriate solution

The problem is a small team shipping a Django and Vue application that must be deployed without visible downtime, on infrastructure that has to be reproducible because each of us works in a separate AWS Academy account.

We chose **Docker Swarm on three EC2 instances** rather than EKS. Swarm gives us replicated services, rolling updates, secrets, configs and placement constraints, which is every primitive this application actually needs, and its control plane can be read in an afternoon. EKS would have added cost and configuration surface for capabilities we would not use. This is the guest lecture's "pragmatic architecture over cloud hype" applied to our own scale.

We chose **blue-green over rolling deployment for production** because the requirement is a verifiable cutover rather than a gradual one. With blue-green the new release is fully deployed and provably healthy on its own ALB listener while the old release still serves every user, so the decision to switch is made against evidence rather than hope. Recovery is then a listener weight change rather than a redeploy, because the old colour was never stopped.

We chose **CloudFormation for all infrastructure and Ansible for all configuration**, with no manual console work in the deployment path, because AWS Academy issues each student a separate account: the same environment had to be stood up independently in every one of them, from the repository alone.

## 6.4 Failure recovery, rollback and scalability

### 6.4.1 Measured behaviour

Every claim below comes from sampling the public endpoint with `curl` every six seconds for the whole of each episode, recording the HTTP status and the release that `/api/version/` reported.

| Episode | Samples | Non-200 responses |
|---|---|---|
| Cutover, pre-merge branch | 31 | 0 |
| Automatic rollback drill | 22 | 0 |
| Three consecutive unplanned failures | 220 | 0 |
| Cutover, re-validated on the merged branch | 114 | 0 |

**Table D.** *Measurement statistics*



**Cutover.** Release `a71dea9` was replaced by `8356851` on the live endpoint. All 31 samples returned HTTP 200 and the reported version changed between two adjacent samples six seconds apart, with no window in which the endpoint was unavailable or served a mixture of releases.

**Automatic rollback.** A release was deployed to the idle colour, verified on that colour's test listener and promoted, then live verification failed deliberately. The pipeline moved the listener back to the previous colour, re-verified the endpoint against all six health checks, sent the alert in Figure C6 and failed the build. The failing release was live for approximately 43 seconds (±6 s, the sampling interval), and all 22 samples returned HTTP 200. That is the honest reading of the result: the failing release was serving correctly, so blue-green rollback did not reduce downtime here, because there was none. What it buys is that recovery is a listener change taking under a minute instead of rebuilding and redeploying the previous release.

**An unplanned failure sequence.** The most convincing measurement was not one we designed. Re-validating the pipeline after the team's branches were merged, we sampled the endpoint continuously for 23 minutes across a run of genuine failures: a build that failed on parameter validation, a second that failed on a missing CloudFormation output, and a production stack update that failed on a missing cross-stack import and was rolled back by CloudFormation. All 220 samples returned HTTP 200 and reported the same release. Each failure occurred at a gate upstream of the traffic switch, which is what made three separate failures invisible to users.

**Re-validated on the merged branch.** The measurements above were taken on the branch where the cutover was developed, so we repeated a full cutover on the integrated branch being submitted, after the pipeline had been restructured and two CloudFormation stacks updated. Release `8356851` on blue was replaced by `065d6c0` on green: sixteen stages, twelve endpoint checks across the idle and live verifications, and 114 samples with no failed request and a clean version change between two adjacent samples.

That run also settled the third defect below. The deployment it performed was a rolling update into a colour *already serving a different release*, the case that had previously passed the replica gate in nineteen seconds while the old release still answered. With the gate now reading Swarm's own roll-out status, the stage waited and the version check that followed it passed for the first time.

### 6.4.2 Scalability

Both colours run two backend and two frontend replicas, spread across nodes by `node.labels.availability_zone`, so a single node or availability-zone loss leaves service capacity in place. Replica counts are pipeline variables, not literals, so scaling out is a variable change and a redeploy. The application is stateless by construction: user-uploaded and seeded media go to a shared S3 bucket rather than container filesystems, and PostgreSQL is external in RDS, which is what makes replicas interchangeable and blue-green possible at all.



### 6.4.3 Challenges, and how they were addressed

Three defects had to be fixed before any of the above worked. All three shared a shape worth reflecting on: **each was a check that reported success without checking anything.**

1. The Ansible role installed the Docker SDK into a virtual environment but verified it with the system interpreter, which cannot see that environment. The task failed on all three nodes. Fixed by verifying with the interpreter the modules actually use (`1b0ea65`).
2. Seeding passed credentials through Ansible's `environment:` block, which sets variables for the `docker service create` process on the manager rather than inside the container, and the `seed_demo` arguments were split across lines inside a single-quoted `-c` script, so a newline terminated the command early. Swarm rejected the job with no logs. Fixed with a Swarm secret and a one-line command (`33f88e7`).
3. The deployment playbook waited for replicas by counting tasks at `desired-state=running`. During a rolling update Swarm keeps the *previous* tasks in that state until it replaces the first, so the count matched immediately and the playbook reported a finished deployment while the old release still answered. A fresh colour hid it, which is why it survived until we deployed twice into the same colour. Fixed by gating on Swarm's own `UpdateStatus` (`51f6ce1`).

The third is the guest lecturer's "automated pipeline gates" point in negative form: a gate that always passes is worse than no gate, because it is counted as evidence.

The same reasoning produced the last feature we added. Idle-colour verification uses the same script and expected release as the live verification, so a broken image fails *before* any traffic moves and can therefore never reach the rollback code. We added a `ROLLBACK_DRILL` parameter that fails live verification after the switch and changes nothing else, letting recovery be rehearsed on demand against real production. Its alert is labelled `DRILL:` rather than `FAILED:`, because a drill that reads like an outage trains a team to ignore alerts. Until we ran it, our rollback was an assumption.

## 6.5 Automation, efficiency, reliability, scalability, repeatability

**Automation.** Colour selection, migration, guarded seeding, deployment, four independent verification layers, cutover and rollback are all automated: from commit to production traffic the only human action is starting the run, once the lab session's temporary credentials have been refreshed. That refresh is the one step we could not automate, and it is a property of AWS Academy rather than of the design, since in production an instance role or OIDC federation would supply those credentials without a human.

**Efficiency.** Container images are built once and promoted from staging to production using the same IMAGE_TAG, without rebuilding for each environment. RELEASE_VERSION separately records the assigned repository’s commit SHA, allowing the deployed release to be traced back to its source.

**Reliability.** Verification is layered deliberately: from inside the Swarm, then against RDS and S3, then through the idle colour's own ALB listener, then against the public endpoint after the



switch. Each layer can fail for reasons the others cannot detect. Failure in either pipeline sends an email alert, and the production pipeline attempts recovery before it reports failure. The infrastructure also carries a CloudWatch dashboard and five alarms covering per-node CPU, ALB target 5xx responses and target response time, publishing to an SNS topic with a confirmed email subscriber (Figures C7 and C8).

**Media storage is shared without being public.** Product images live in an S3 bucket with all four public-access blocks enabled, and the application serves them as presigned URLs signed by the instance role. We verified this rather than assuming it: the same image path returns HTTP 403 without its signature and HTTP 200 with the full 1,661-character signed query string. Shared media is what makes the two colours interchangeable, and this way the bucket is not world-readable to achieve it. The trade-off is that a signed URL expires with the credentials that signed it, so a page kept open long after it was rendered shows broken images.

**Scalability.** Covered in 6.4: parameterised replicas, AZ-spread placement, stateless application containers, external database and object storage.

**Repeatability.** Our strongest evidence is incidental. Because AWS Academy issues each student a separate account, the entire stack of Jenkins, shared services, staging and production was built from nothing in an account that had never seen it, using only the repository and the setup wizard, and reached a verified production cutover in the same day.

## 6.6 Where the pipeline is still weak

An honest self-evaluation has to name what we did not solve.

**Both colours share one database.** This is the most important limitation. A backward-incompatible migration can break the colour that is still serving, and a traffic rollback does not undo a schema change, so blue-green as we built it protects against application defects but not schema defects. We snapshot the database before migration and built a restore-rehearsal pipeline, but the rollback path itself does not include the database.

**We have metrics, not observability.** CloudWatch metrics and email alerts tell us that something failed. The guest lecture named distributed tracing (OpenTelemetry, Datadog) and structured incident workflows as the practice, and we have neither: diagnosing a production problem still means reading Swarm logs by hand.

**Much of our application testing was added after the initial deployment work.** The current application suite contains 24 backend test functions, three frontend cases and two end-to-end scenarios, backend parametrisation expands the overall total to 31 cases, excluding recovery tests. Seven backend modules were authored on 8 September, while additional backend tests and the checkout end-to-end scenario were committed on 9 September. These additions came after the fixes for the three deployment defects discussed in 6.4. The coverage is useful, but its timing meant it could not protect those earlier development iterations. Moreover, the application tests do not directly exercise the interpreter configuration, container credential delivery or rollout-waiting logic responsible for those defects. End-to-end tests may expose their symptoms, but targeted deployment-tooling tests and deployment verification checks are needed to address those failure modes.



**The drill injects the failure rather than a fault.** `ROLLBACK_DRILL` proves the recovery path executes correctly; it does not prove the pipeline detects an arbitrary real fault. Genuine fault injection is the next step.

**We had no review gate.** Pipeline work was integrated by merging branches directly, and all three defects above reached the integration branch that way. The guest lecturer's rule that every change should be accountable to two people is the practice whose absence cost us the most, and the few changes we did route through pull requests cost seconds more than a direct merge.

# 7. Project Responsibilities

| | Member 1 | Member 2 | Member 3 | Member 4 | Member 5 |
|---|---|---|---|---|---|
| **Member Name** | Ma Thanh Long Mason | Do Hao Nhien | Tran Hoang Nguyen | Bui Linh Nhi | Nguyen Tran Nhat Huy |
| **Student ID** | S4042629 | S3994477 | S4054071 | S4086154 | S3978239 |
| **Contribution (%)** | 20% | 20% | 20% | 20% | 20% |

**Table E.** *Contribution table (minimal)*

The full contribution table, which includes a list of tasks for each member, will be available in Appendix E.

# 8. Conclusion

This assignment helped us to further elaborate upon the DevOps technologies and knowledge learned during lectures and tutorials by giving us a chance at a hands-on exercise. Through the assignment, our team has managed to develop a CI/CD pipeline that connects version control, automated testing, and infrastructure provisioning into one unified workflow. Measured against the pain points in 1.1, deployment is no longer manual, testing cannot be skipped, and rollback is a path we have exercised rather than assumed.

Working with Jenkins, Docker, Ansible, and CloudFormation strengthened our skills in working with Infrastructure as Code, and taught us the importance of being able to rebuild an entire environment from the repository alone, which each of us had to do in a separate AWS Academy account.

Three lessons stand out. A check that cannot fail is worse than no check: all three defects in 6.4 were gates that reported success without verifying anything. A claim has to be measured, so we sampled the public endpoint every six seconds through a cutover and a rollback drill. And automation removes manual work, not accountability: skipping code review costs us the most time.



The pipeline is not finished, and 6.6 is exact about where it stops. Being precise about what it covers is more useful than claiming it covers everything.

# References

[1] H. A. Nguyễn, "Guest lecture and Q&A," presented at COSC2767 Systems Deployment and Operations, RMIT University Vietnam, Ho Chi Minh City, Vietnam, Aug. 26, 2026. Quotations are taken from the recording of the session; minor transcription artifacts have been silently corrected and elisions are marked.

# Appendix A: Main Requirements Evidence

## A.1 Cloud Integration Figures

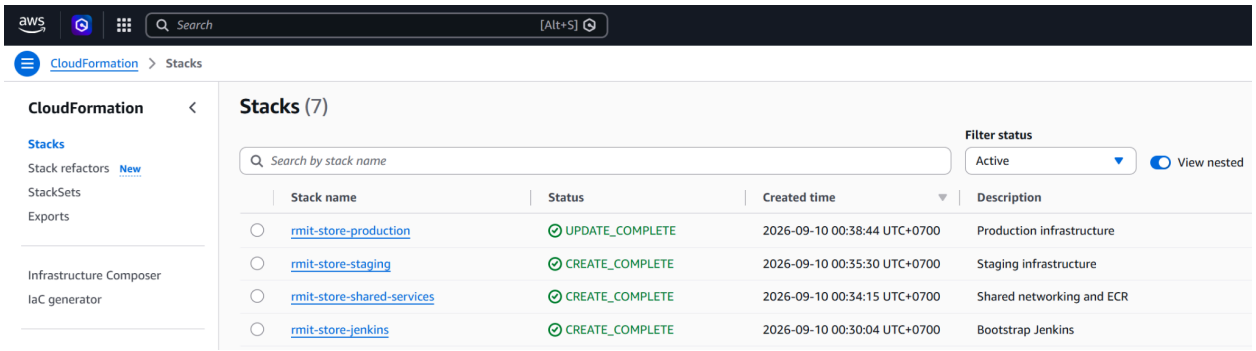



**Figure A1.1.** *Stacks list*

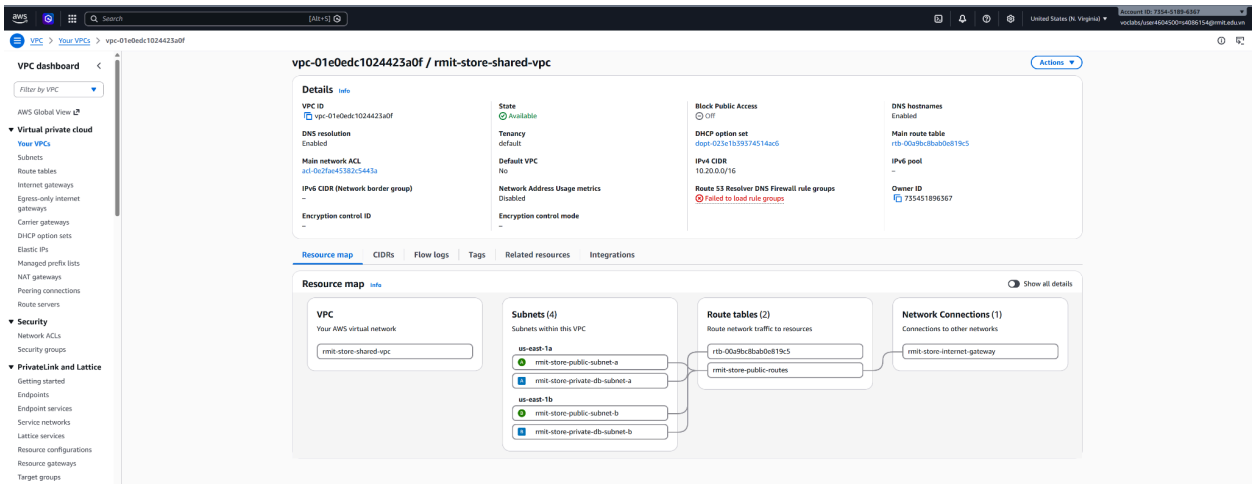



**Figure A1.2.** *VPC with public/private subnets across two AZs*

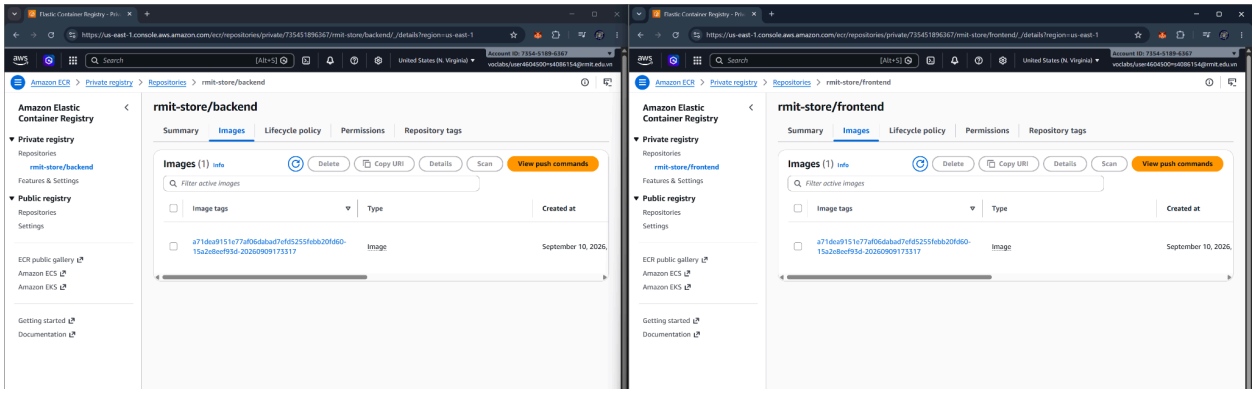



**Figure A1.3.** *Repositories in ECR*

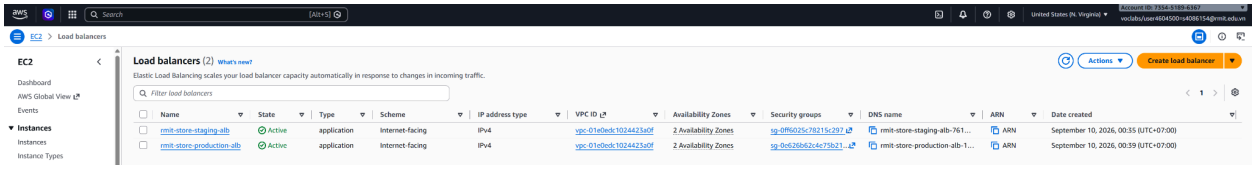



**Figure A1.4.** *Load balancers*

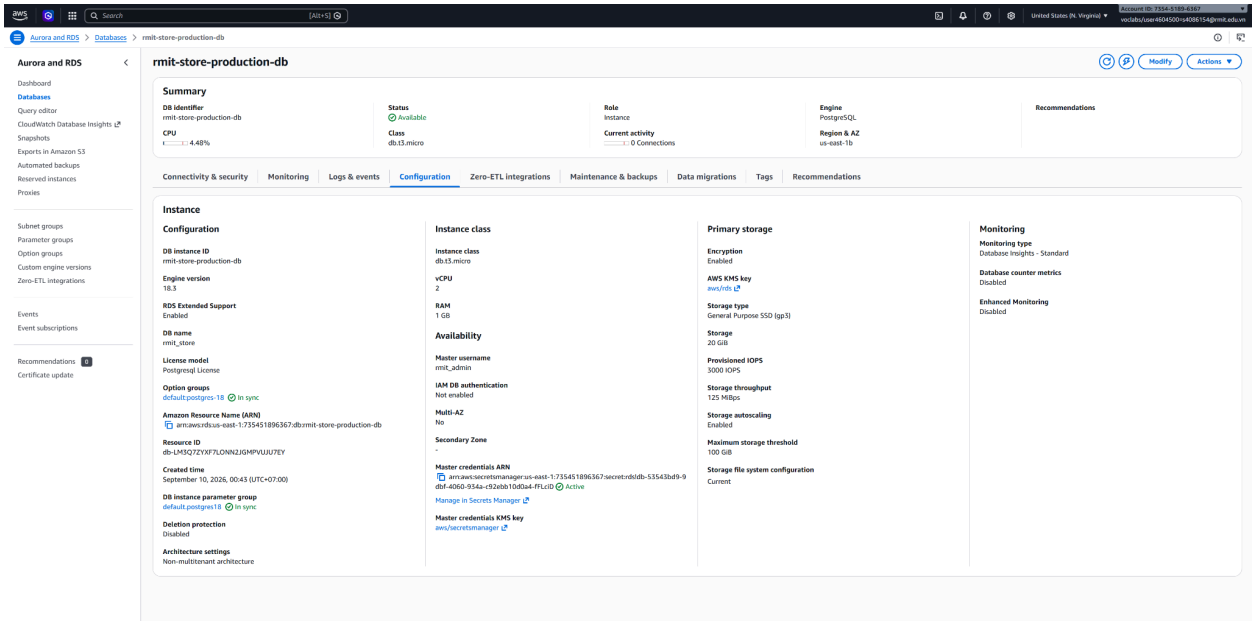



**Figure A1.5.** *RDS database*

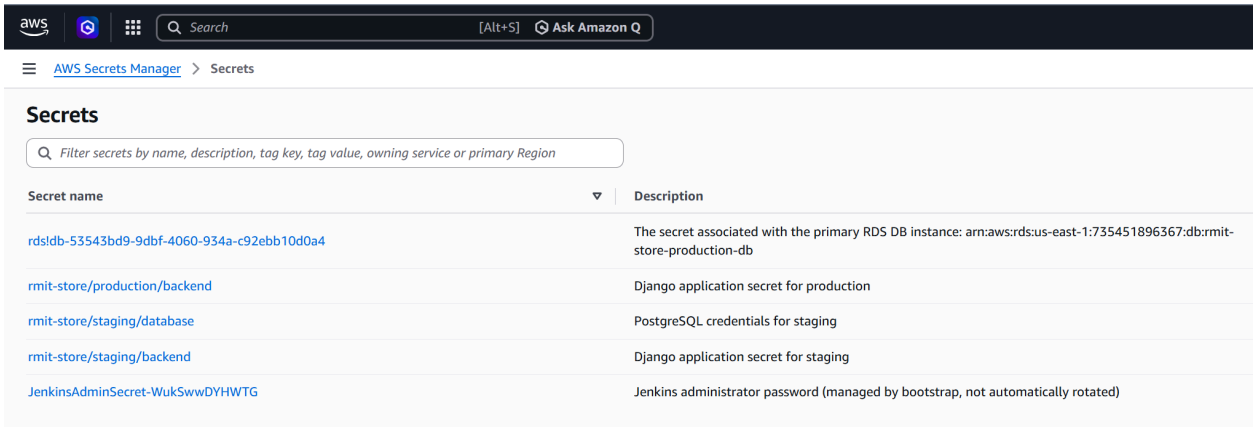



**Figure A1.6.** *Secrets on Secrets Manager*

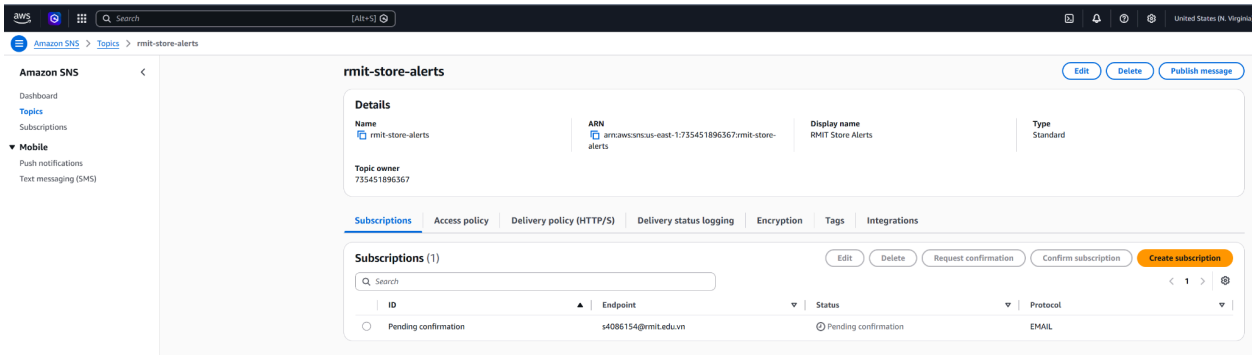



**Figure A1.7.** *rmit-store-alerts SNS topic and subscribed email*

# A.2 Containerised Services

```dockerfile
# RMIT University Vietnam
# Course: COSC2767|COSC2805 Systems Deployment and Operations
# Semester: 2026B
# Assessment: Assignment 2
# Author: Ma Thanh Long Mason
# ID: s4042629
# Created  date: 19/08/2026
# Last modified: 19/08/2026
# Acknowledgement: https://docs.docker.com/build/building/multi-stage/

# STAGE 1 - Building dependencies
FROM node:24-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# STAGE 2 - Configuring nginx
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

**Figure A2.1.** *Frontend Dockerfile*

```dockerfile
# Minimal Dockerfile for Django application with Gunicorn
FROM python:3.14-slim-bookworm

# PYTHONUNBUFFERED: Prevents Python from buffering stdout and stderr
# PIP_NO_CACHE_DIR: Disables caching for pip
ENV PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

# Start
WORKDIR /app

# Creating new user and group
RUN addgroup --system --gid 10001 appgroup \
    && adduser --system --uid 10001 --ingroup appgroup appuser

# Installing dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Give ownership to the group and user
COPY --chown=appuser:appgroup . .
RUN chmod +x /app/start.sh \
    && mkdir -p /app/media /app/staticfiles \
    && chown -R appuser:appgroup /app/media /app/staticfiles

USER appuser

EXPOSE 8000

# Start using start.sh bashscript
ENTRYPOINT ["/app/start.sh"]

```

**Figure A2.2.** *Backend Dockerfile*

```bash
#!/bin/bash

# RMIT University Vietnam
# Course: COSC2767|COSC2805 Systems Deployment and Operations
# Semester: 2026B
# Assessment: Assignment 2
# Author: Ma Thanh Long Mason
# ID: s4042629
# Created  date: 19/08/2026
# Last modified: 19/08/2026
# Acknowledgement: Acknowledge the resources that you use here.

# -e: Stop if command fails
# -u: Stop if variable is unset
set -eu

# Collect static files (Django command)
python manage.py collectstatic --noinput

# Run whatever command is passed through the command line (e.g. gunicorn)
exec "$@"
```

**Figure A2.3.** *Backend Entryfile*

```
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Sep 14 10:35:03 2026 from 35.171.139.126
[ec2-user@staging-node ~]$ sudo docker stack services rmit-store-staging
ID             NAME                          MODE         REPLICAS   IMAGE
                                   PORTS
iynp36pkfelt   rmit-store-staging_backend     replicated   1/1        575187244258.dkr.ecr.us-east-1.amazonaws.com/rmit-store/backend:53a252c719c61215a5b8a79180725e03bc60f0af-53
a252c719c6-20260914103017
ynu3tn9hxsks   rmit-store-staging_db          replicated   1/1        postgres:16-alpine

gbe6pb1bud79   rmit-store-staging_frontend    replicated   1/1        575187244258.dkr.ecr.us-east-1.amazonaws.com/rmit-store/frontend:53a252c719c61215a5b8a79180725e03bc60f0af-5
3a252c719c6-20260914103017
[ec2-user@staging-node ~]$
```

**Figure A2.4.** *Staging node docker services*

# A.3 Continuous Integration

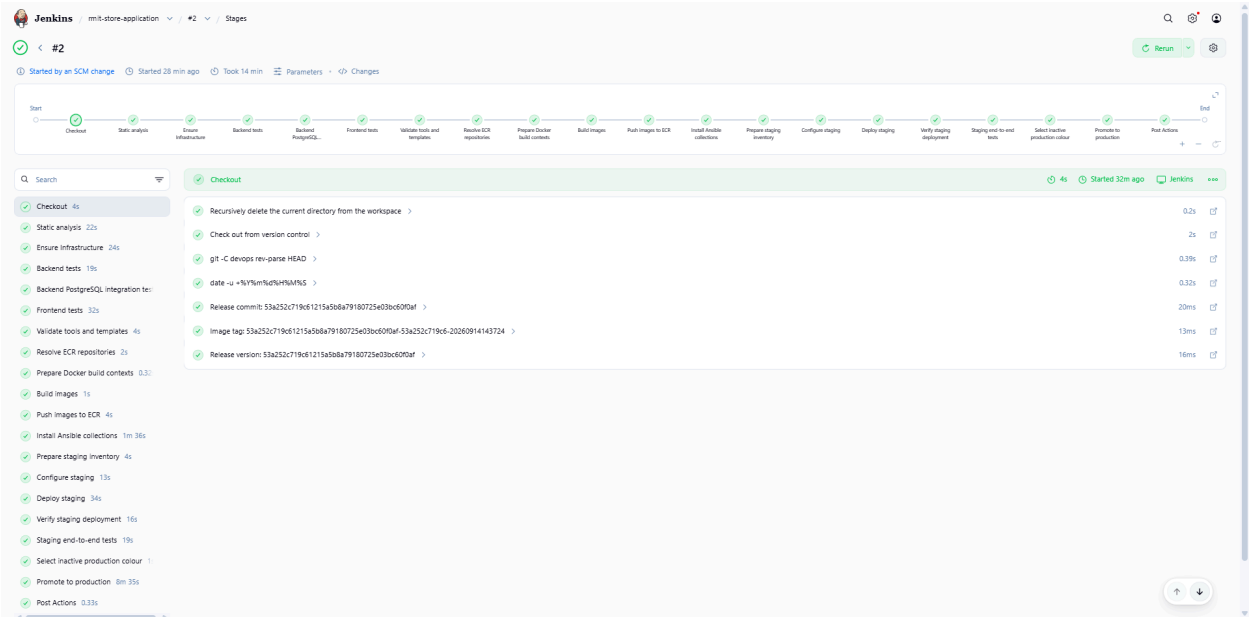



**Figure A3.1.** *Successful CI stages in the Jenkins application pipeline*

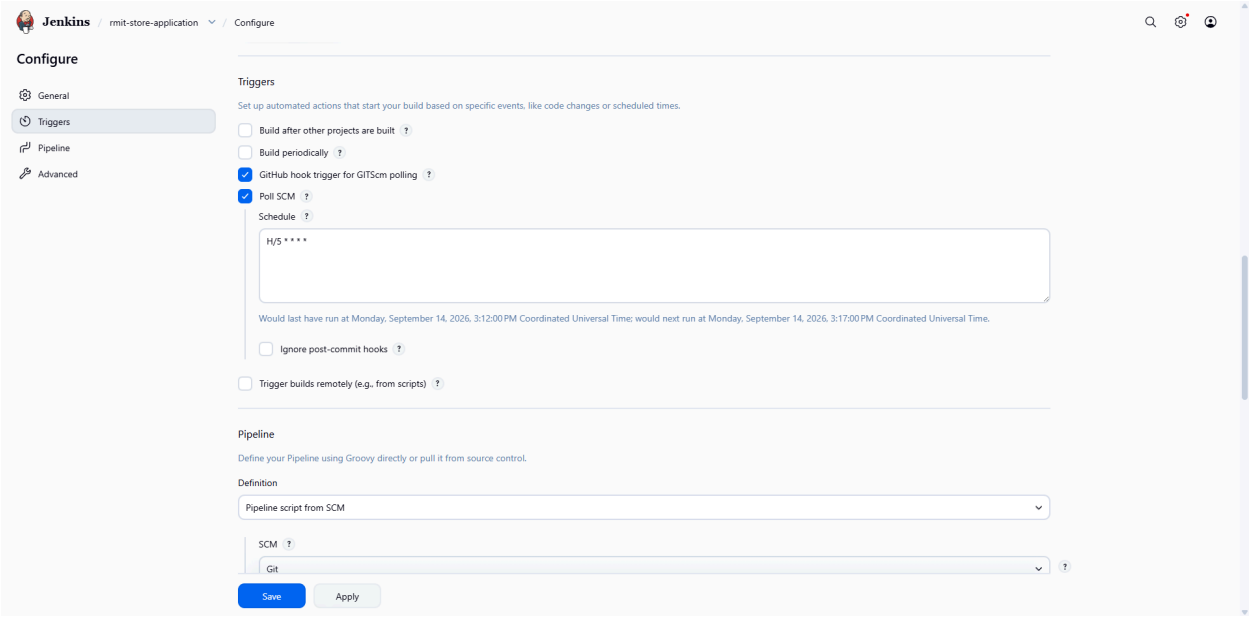



**Figure A3.2.** *GitHub push and a 5 minute SCM poll automatically triggering the Jenkins application pipeline*

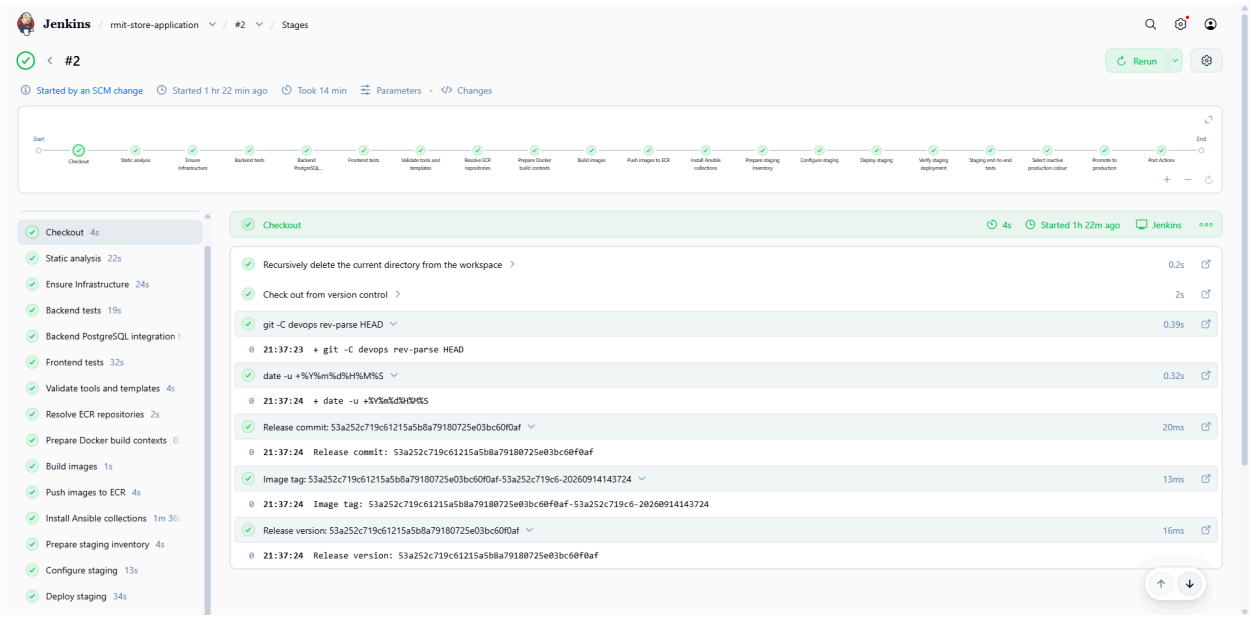



**Figure A3.3.** *Checkout*

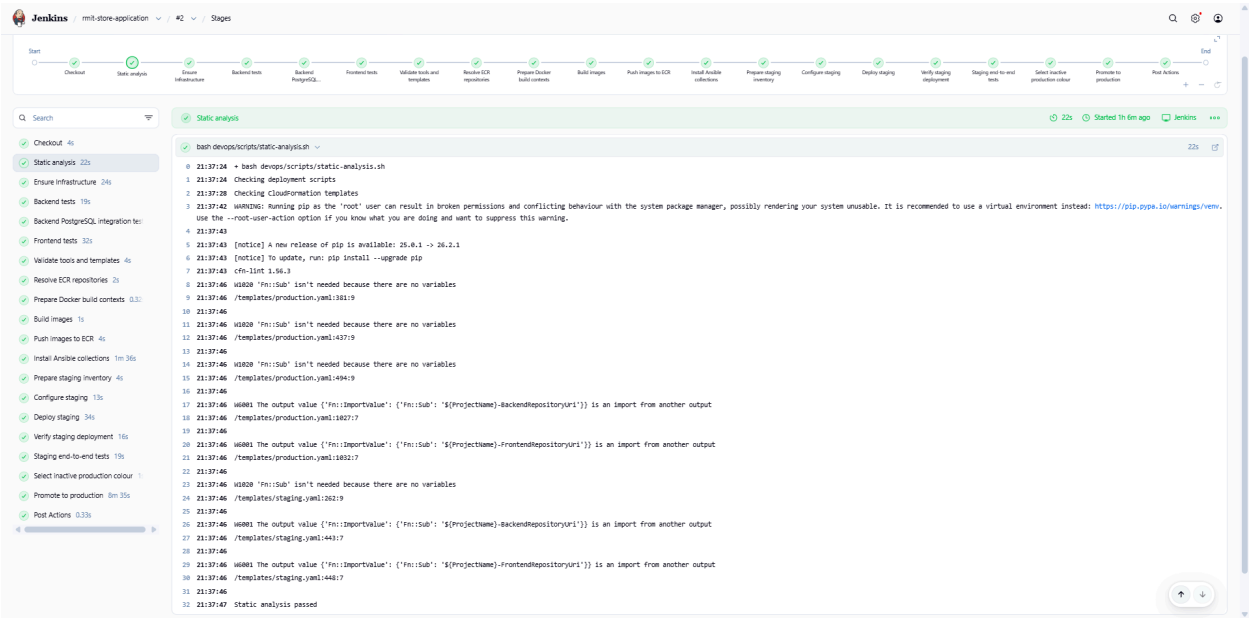



**Figure A3.4.** *Static analysis*

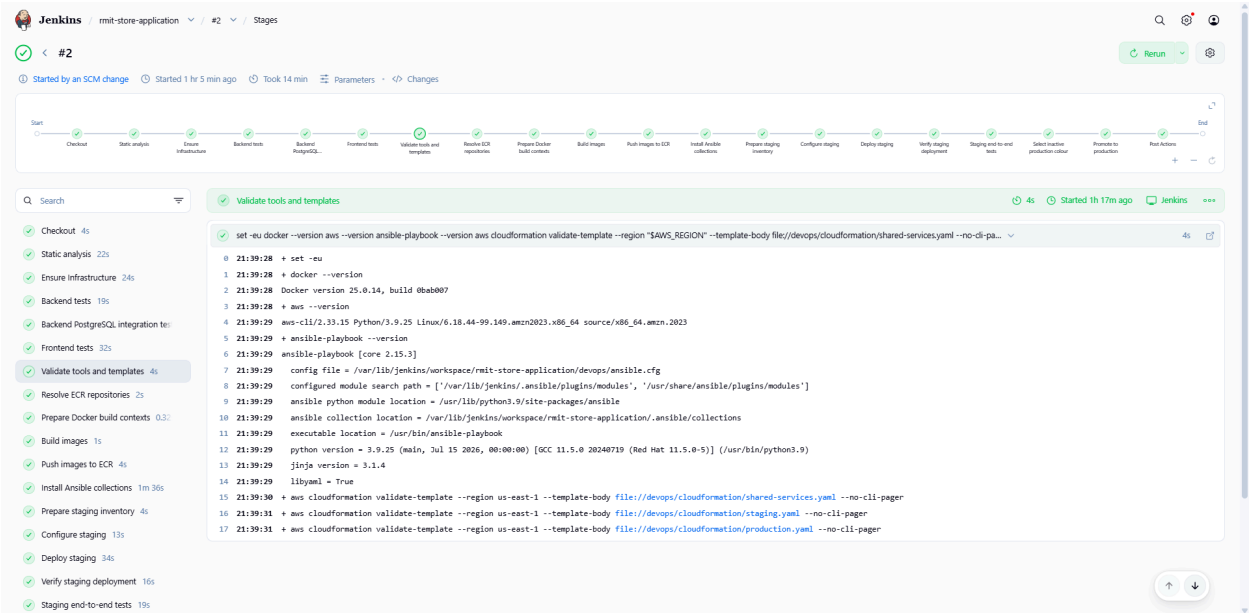



**Figure A3.5.** *Validate tools and templates*

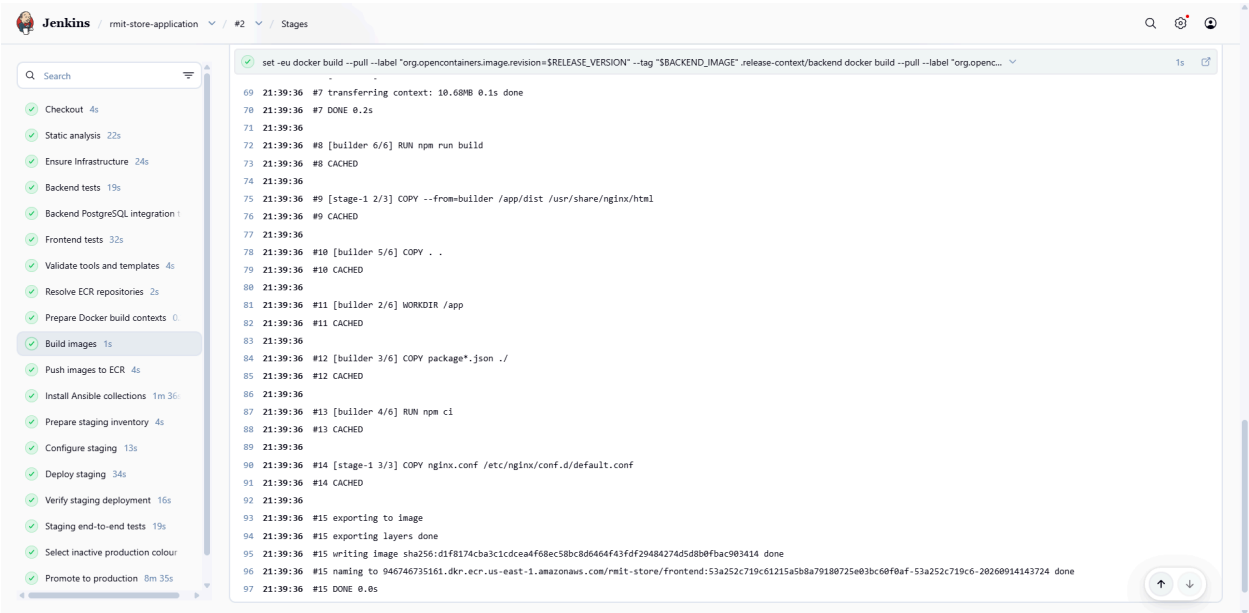



**Figure A3.6.** *Build images with the immutable tag*

# A.4 Continuous Deployment

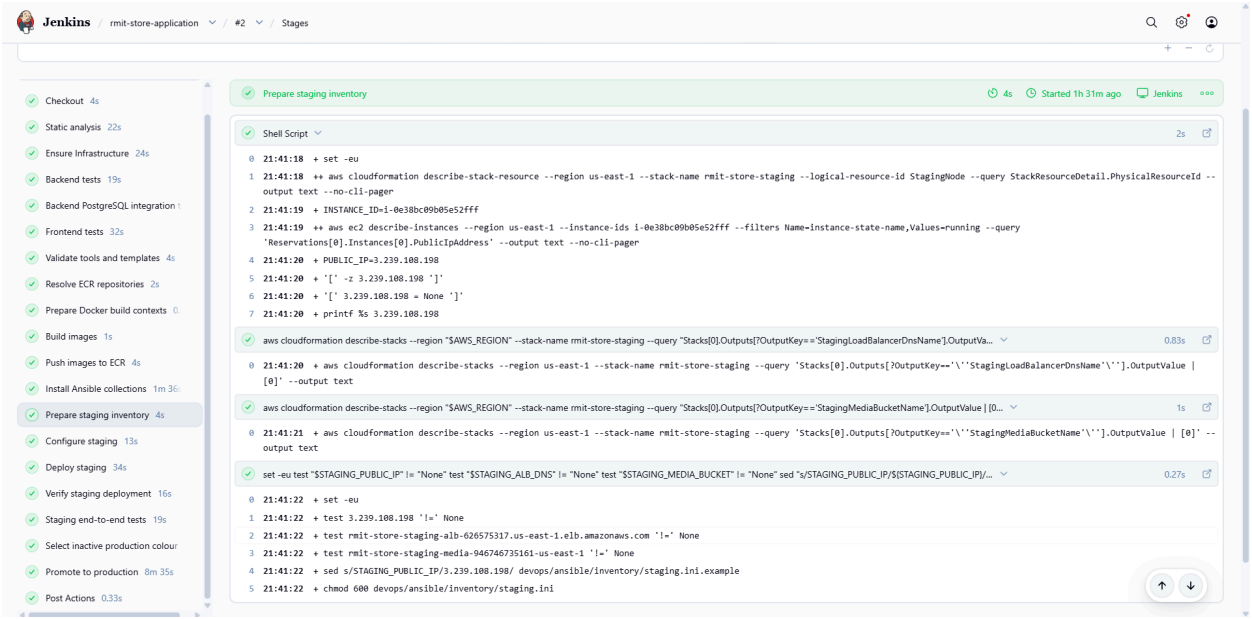



**Figure A4.1.** *Prepare staging inventory*

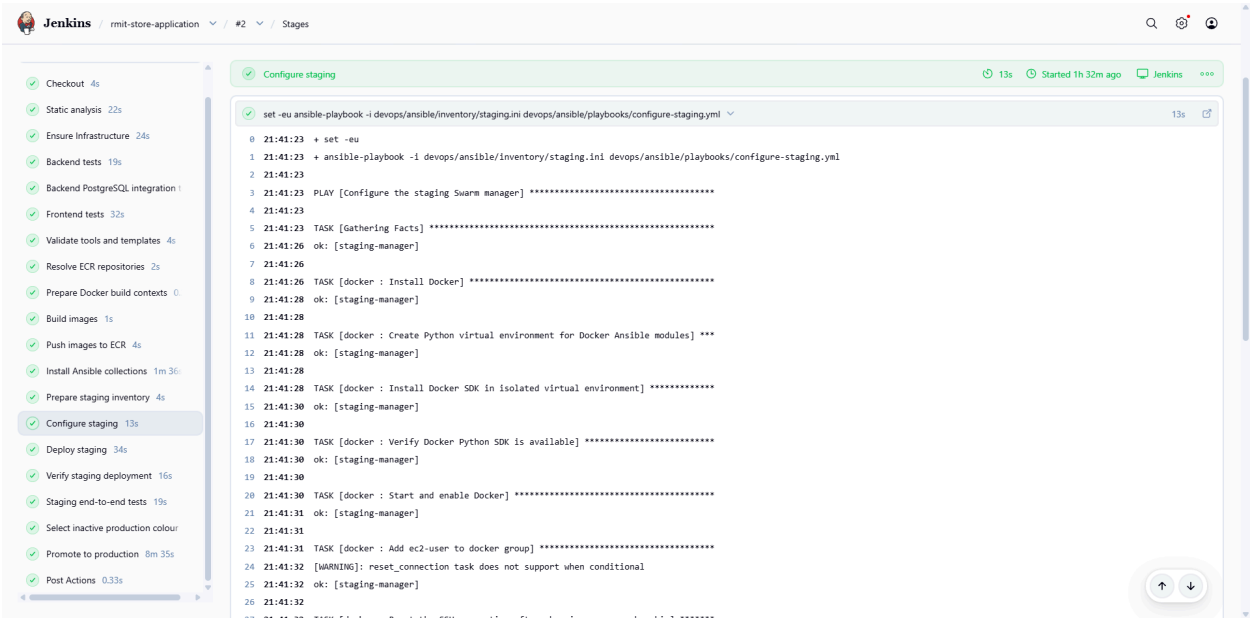



**Figure A4.2.** *Configure staging*

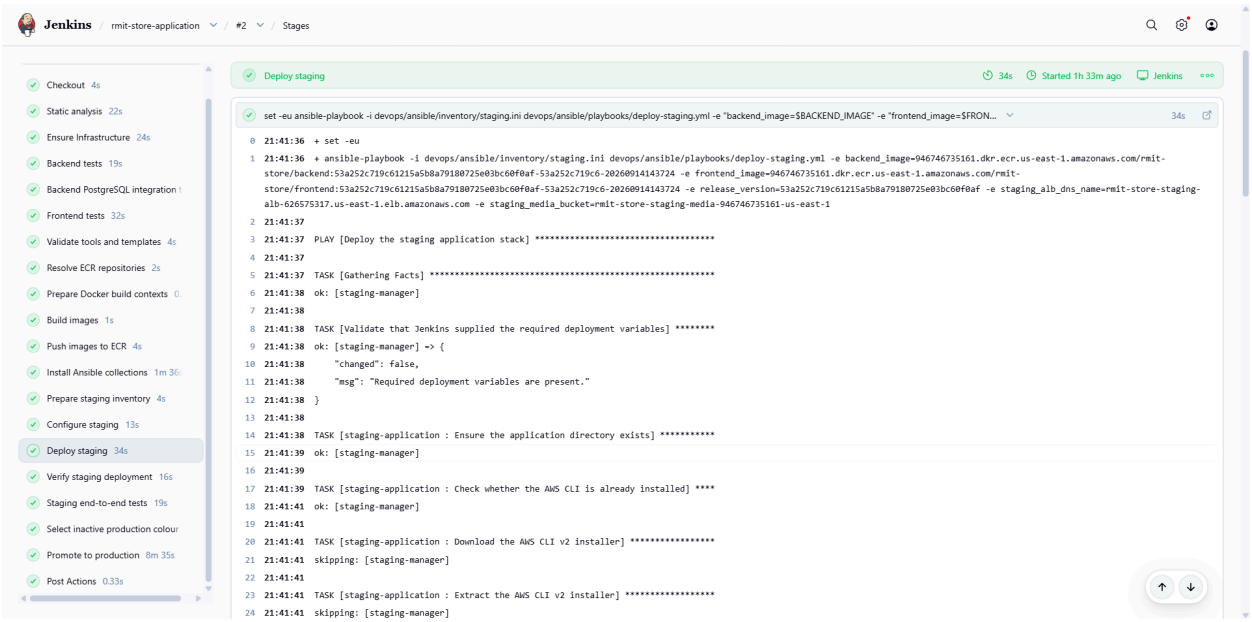



**Figure A4.3.** *Deploy staging*

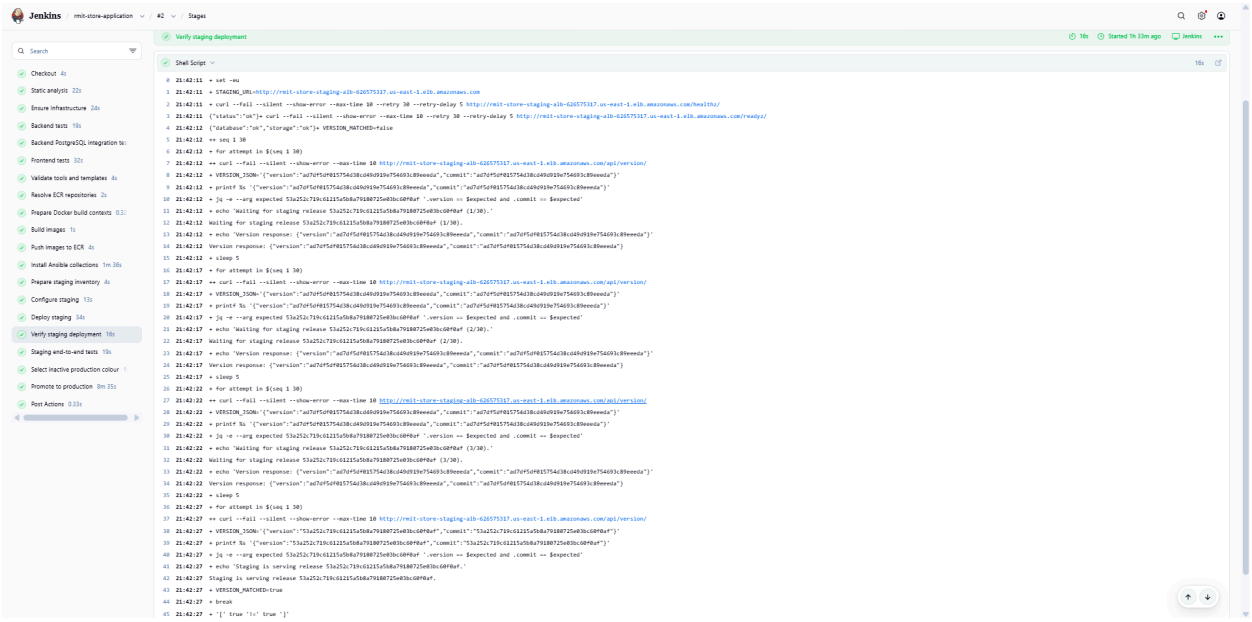



**Figure A4.4.** *Verify staging deployment*

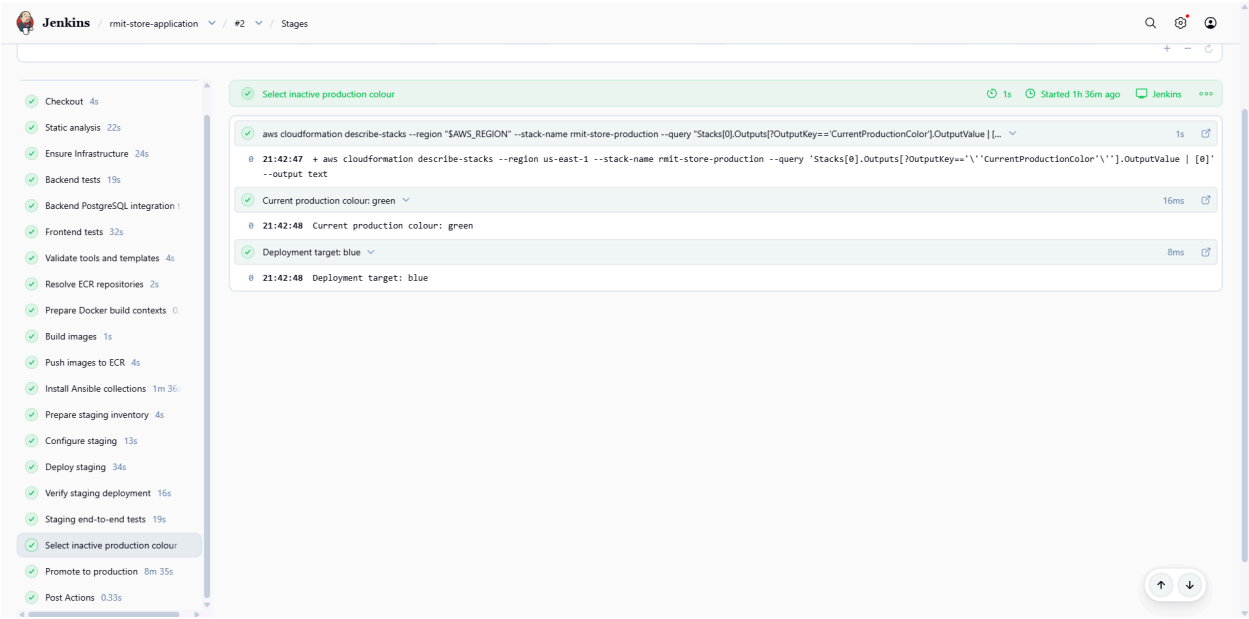



**Figure A4.5.** *Select inactive production colour*

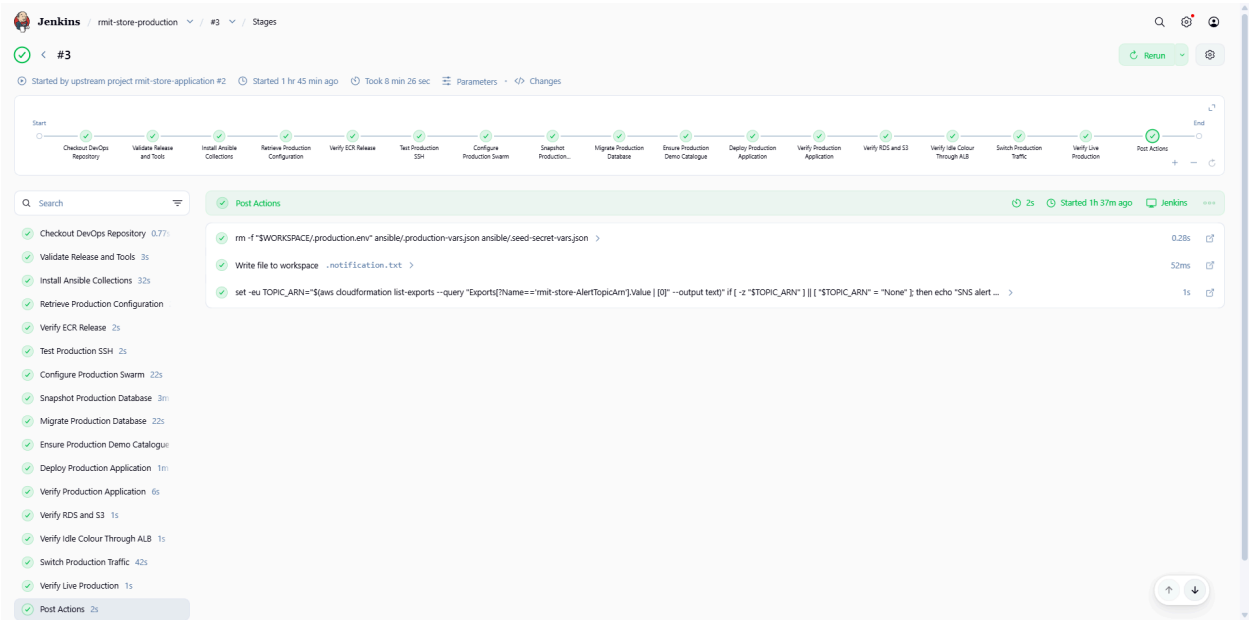



**Figure A4.6.** *Successful CD stages in the Jenkins production pipeline*

# A.5 Testing Framework

## A.5.1 Figures

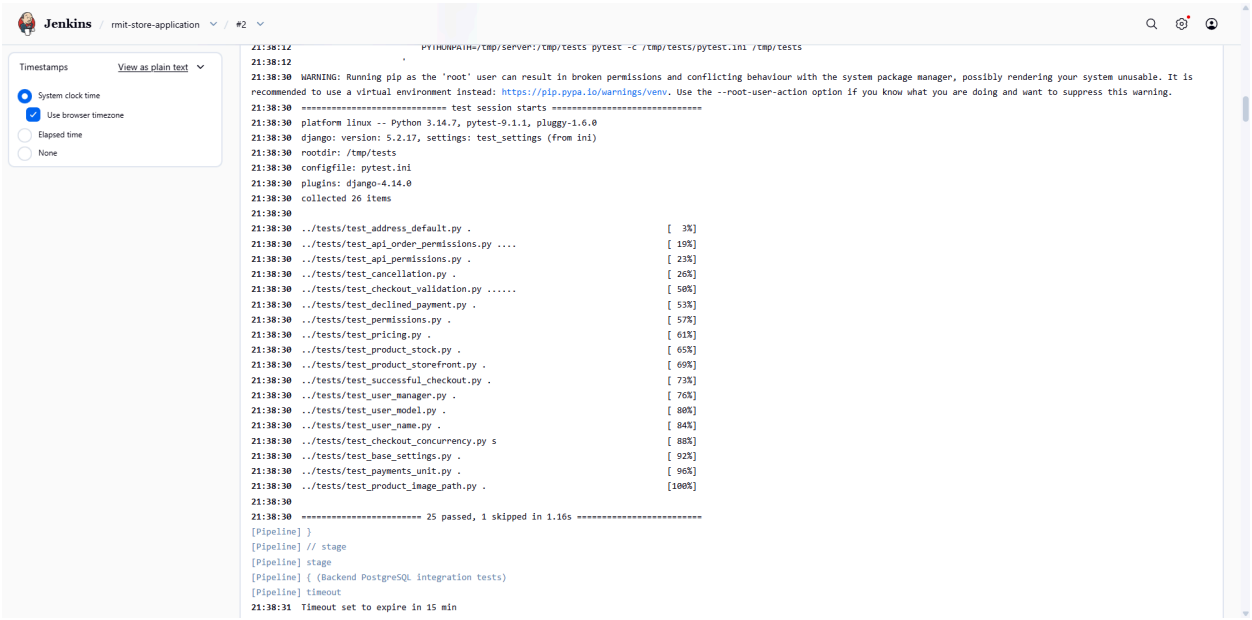



**Figure A5.1.** *Backend tests*

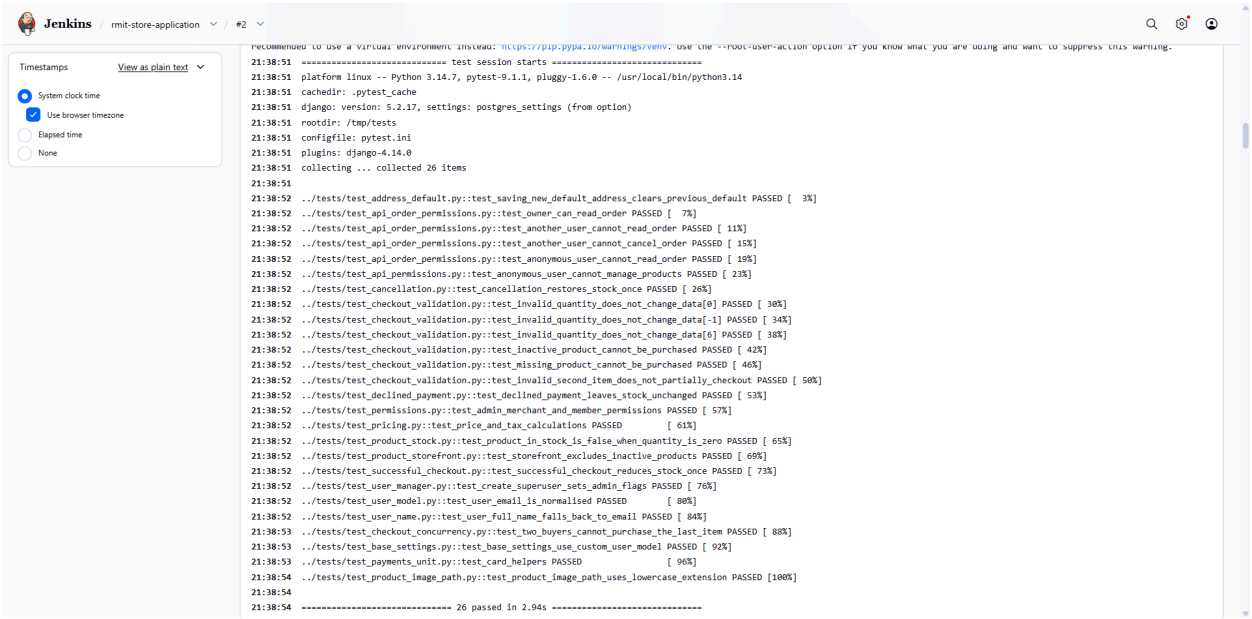



**Figure A5.2.** *PostgreSQL, Backend Integration tests*

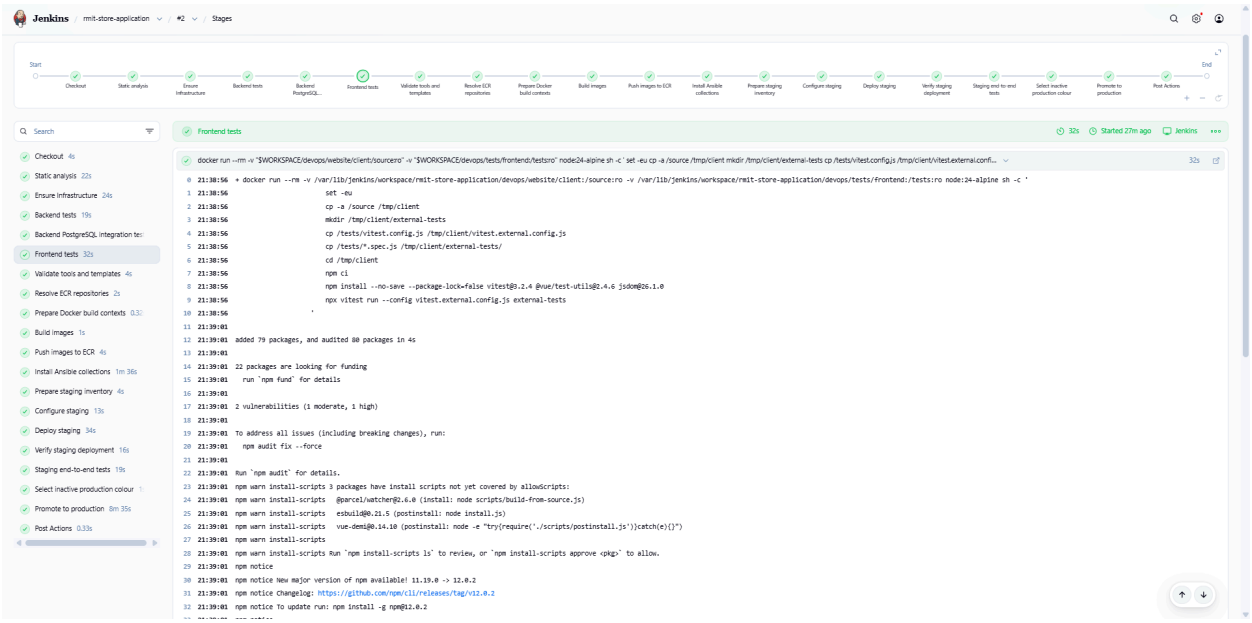



**Figure A5.3.** *Frontend tests 1*

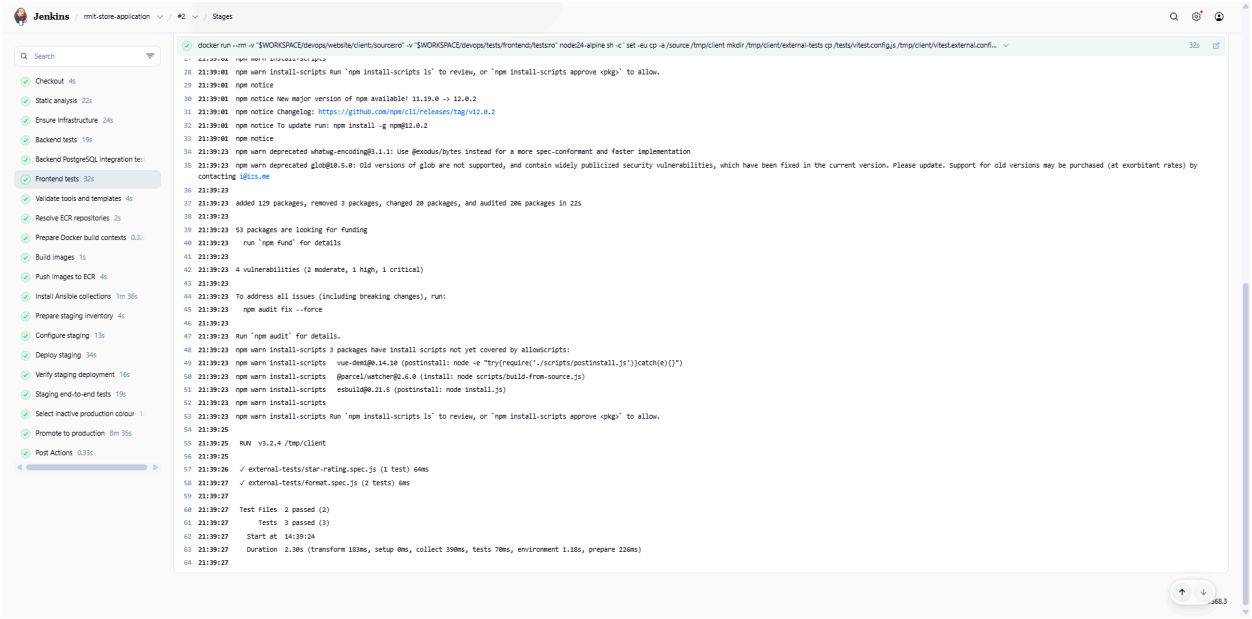



**Figure A5.4.** *Frontend tests 2*

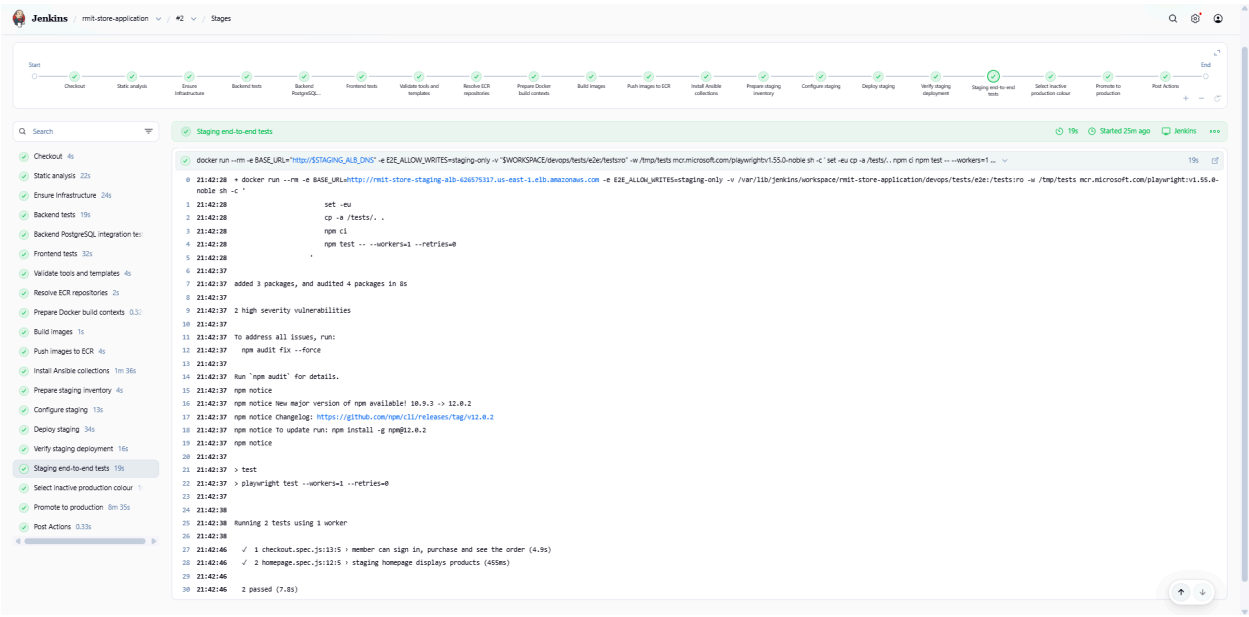



**Figure A5.5.** *End-to-end test*

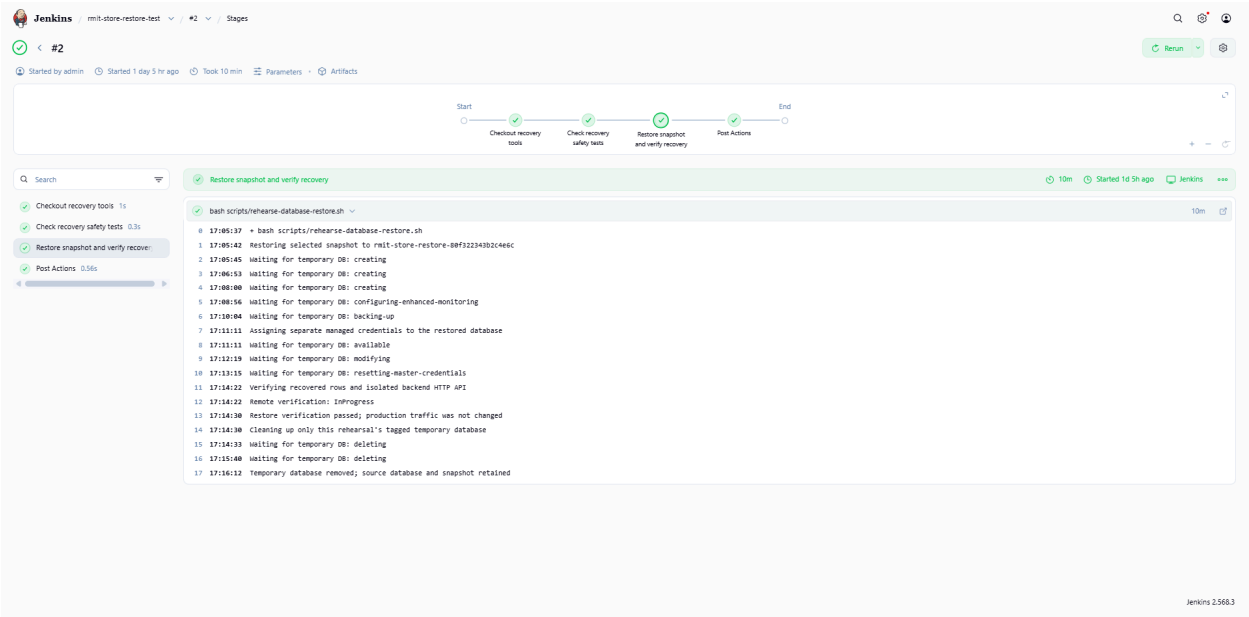



**Figure A5.6.** *Restore Test*

## A.5.2 Tests list

### Backend Tests

- **test_payments_unit.py:** checks card-number normalization, Luhn validation, and card-brand detection without using a database or HTTP.
- **test_user_model.py:** verifies email normalization and password verification.



- **test_user_manager.py:** confirms that created superusers receive administrator, staff, and superuser privileges.
- **test_user_name.py:** checks that a user without a full name falls back to their email address.
- **test_base_settings.py:** confirms that Django uses the custom accounts.User model.
- **test_permissions.py:** verifies admin and merchant permissions, including rejection of inactive merchants.
- **test_api_permissions.py:** ensures anonymously GET to the product-management listing, users cannot access product-management endpoints.
- **test_api_order_permissions.py:** confirms that order owners can view their orders, while other users and anonymous users cannot. It also ensures unauthorized users cannot cancel orders.
- **test_address_default.py:** ensures that a user can have only one default address.
- **test_product_stock.py:** verifies that products with zero quantity are reported as out of stock.
- **test_product_storefront.py:** ensures inactive products are hidden from the storefront.
- **test_product_image_path.py:** checks that product image paths use the correct directory and lowercase file extension.
- **test_pricing.py:** verifies product subtotals, tax calculations, rounding, and final order totals.
- **test_successful_checkout.py:** confirms that successful checkout reduces stock correctly and combines duplicate products into one order item.
- **test_declined_payment.py:** ensures declined payments do not create orders or reduce stock.
- **test_checkout_validation.py:** checks invalid quantities, inactive products, missing products, and multi-item checkout failures. It also verifies that failed checkouts do not partially modify the database.
- **test_cancellation.py:** confirms that cancellation restores stock, marks the order as cancelled, records a refund, and does not restore stock twice.
- **test_checkout_concurrency.py:** simulates two customers purchasing the final item and confirms that only one order succeeds.

### Frontend Tests

- **format.spec.js:** verifies that utility functions produce uppercase initials and correctly extract API field errors.
- **star-rating.spec.js:** confirms that selecting a rating emits the expected Vue update:modelValue event.

### End-to-End Tests

- **homepage.spec.js:** verifies that the deployed staging homepage loads and displays products.
- **checkout.spec.js:** tests the complete customer journey: registration, login, product selection, payment, order creation, viewing the order, and cleanup by cancelling the test order.

### Recovery Tests

- **test_rehearsal.py:** tests recovery input validation, snapshot validation, cleanup ownership, AWS command safety, image selection, timeouts, missing credentials, remote failures, and cleanup failures.
- **verify_restored_database.py:** verifies that the restored database is read-only, contains the expected schema and records, preserves order totals, starts successfully, and serves catalogue data through the real API.

The backend normally uses an in-memory SQLite database configured in test_settings.py. The concurrency test uses PostgreSQL through postgres_settings.py, since PostgreSQL is required to test database row locking accurately.

## A.6 Automation Tool

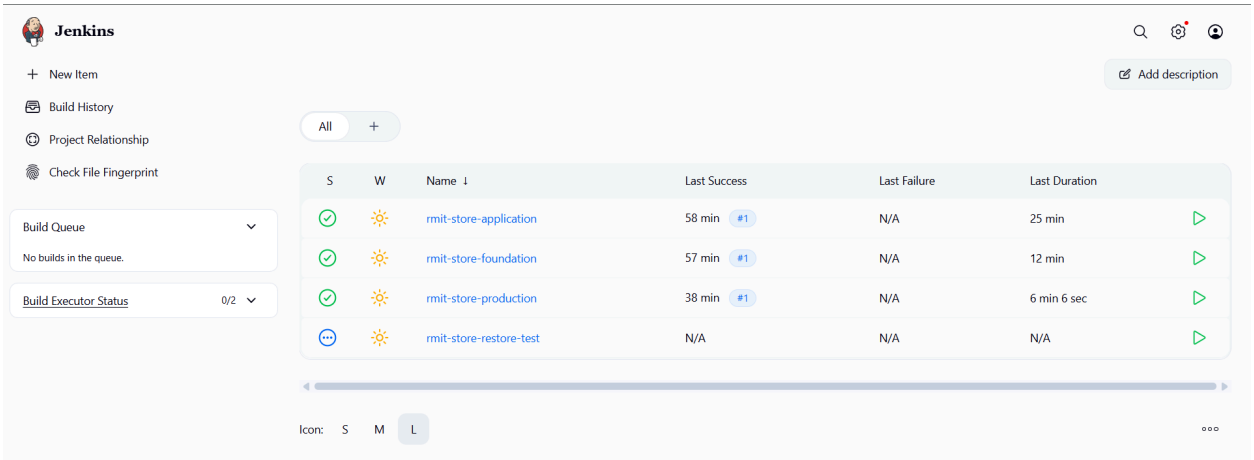



**Figure A6.1.** *Jenkins jobs list*

# A.7 Configuration Management

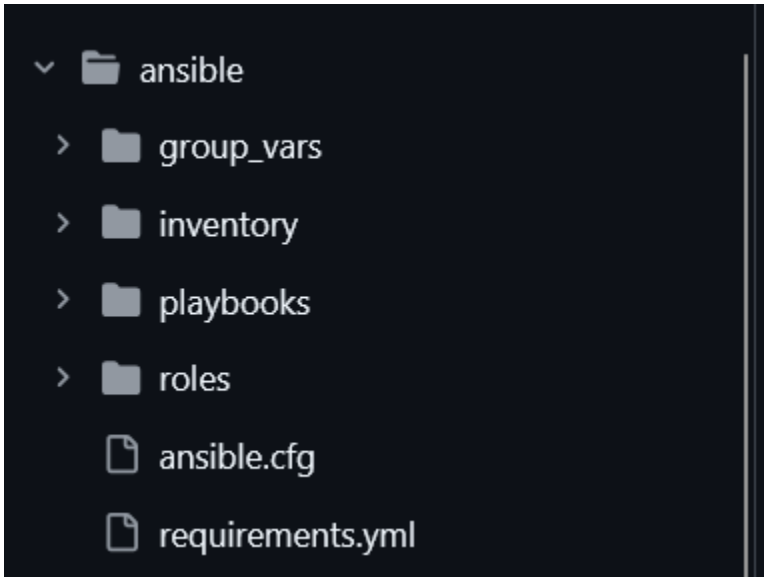



**Figure A7.1.** *Ansible collections*

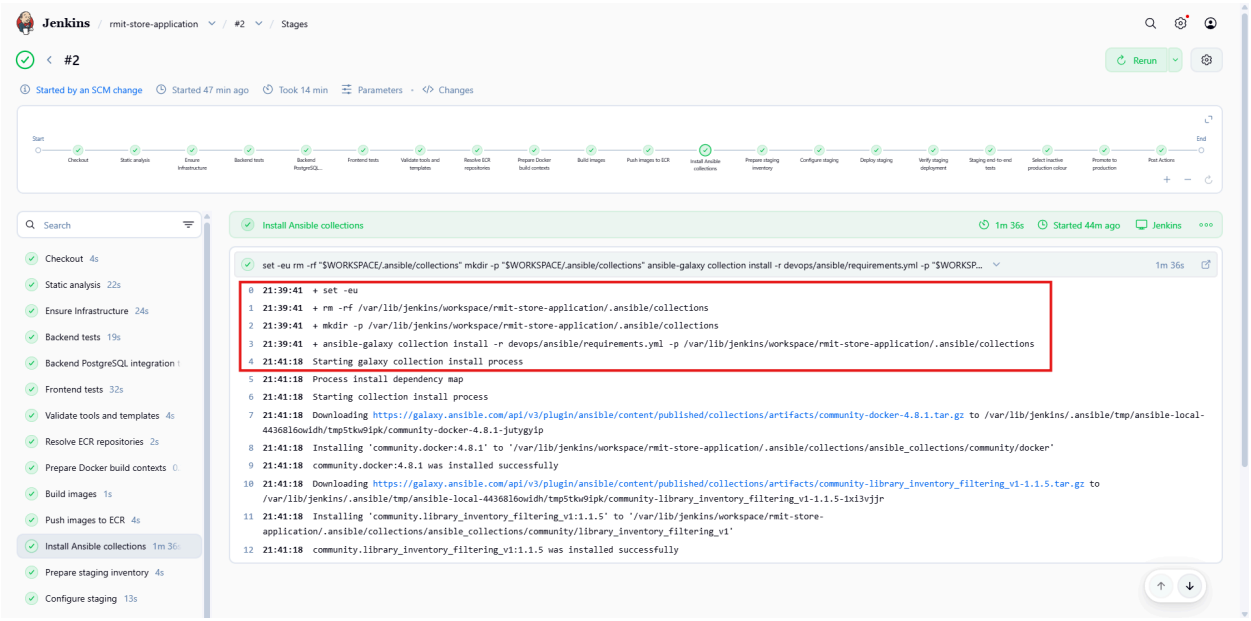



**Figure A7.2.** *Ansible collections install*

# A.8 Automated Alerting

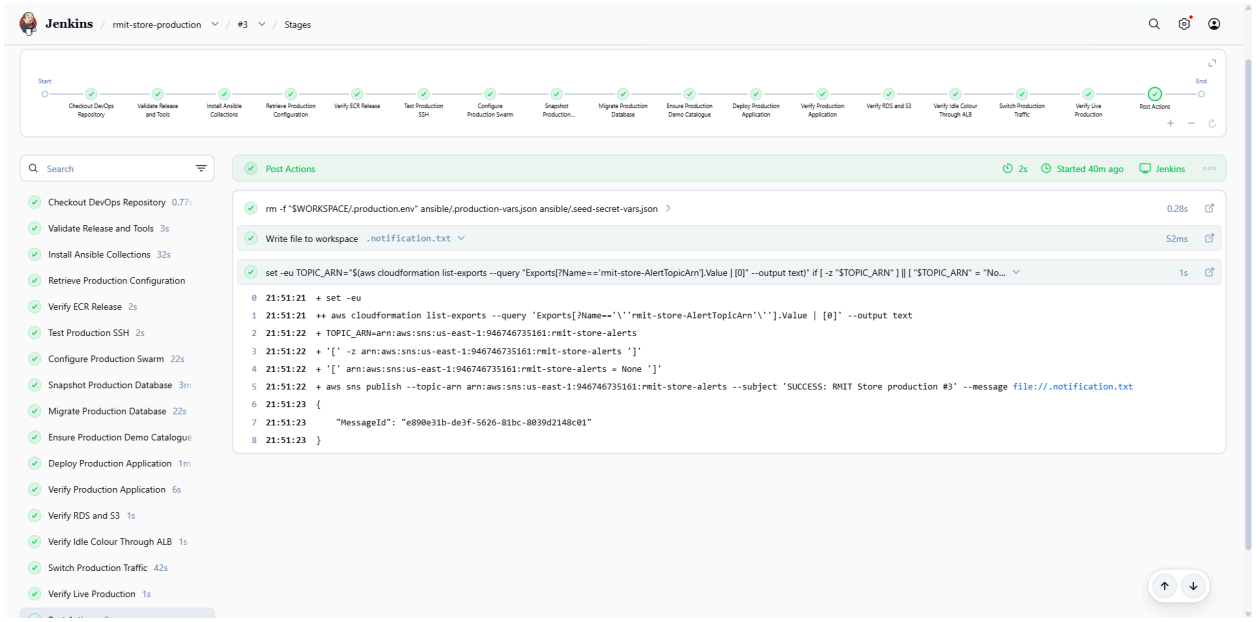



**Figure A8.1.** *Sending SNS email*

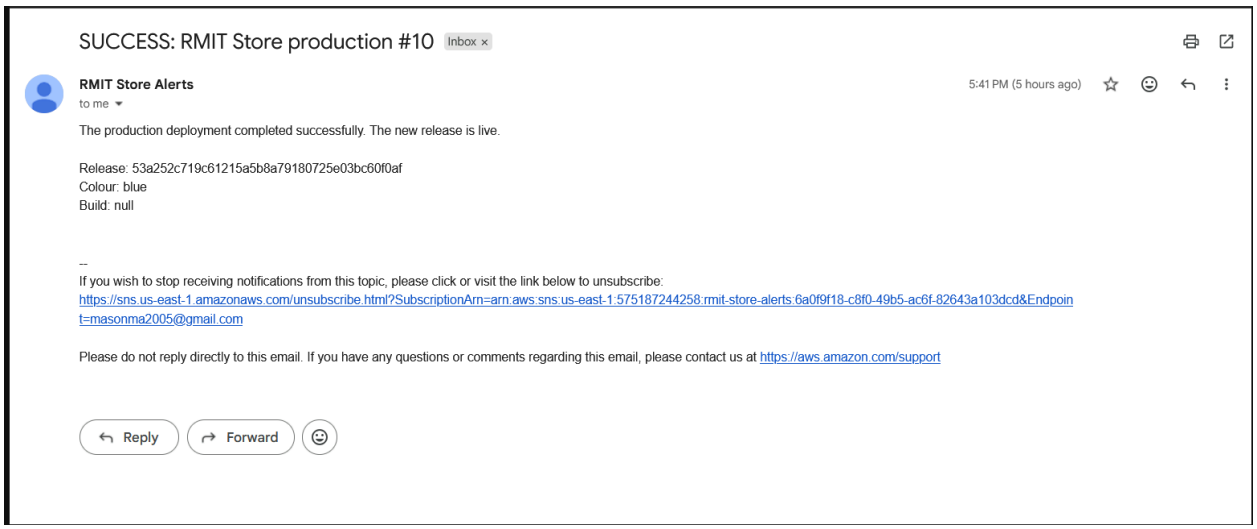



**Figure A8.2.** *Success Email*

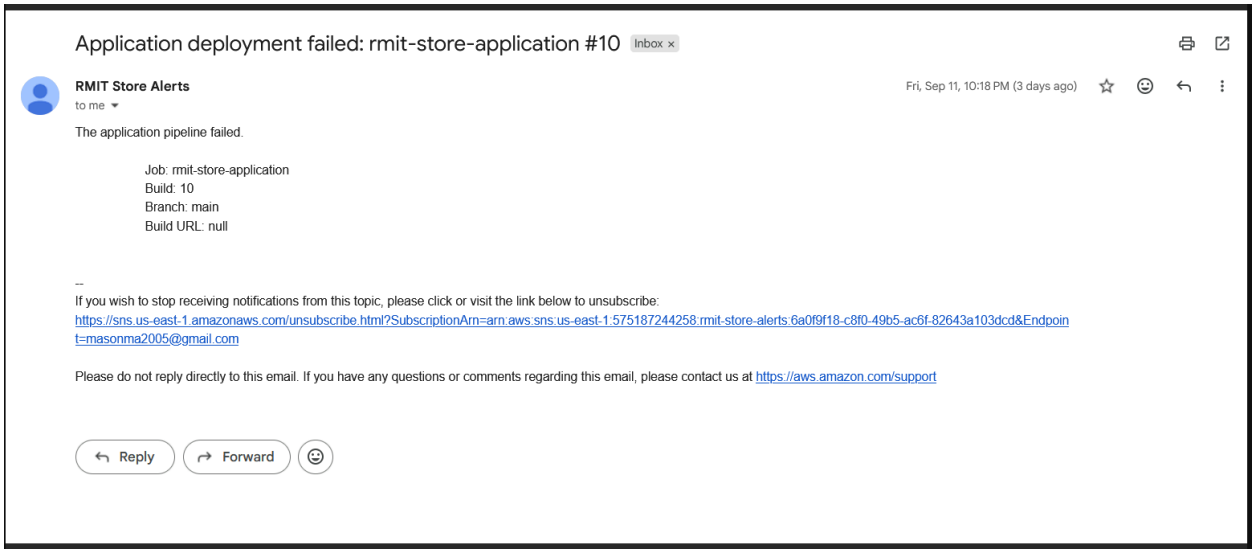



**Figure A8.3.** *Failure Email*

# A.9 Orchestration

```
[ec2-user@staging-node ~]$ docker node ls
ID                            HOSTNAME        STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
k15w48wh9xfya6wbobj1zf0ba *   staging-node    Ready     Active         Leader           25.0.16
```

**Figure A9.1.** *Staging Docker swarm*

```
[ec2-user@production-manager ~]$ docker node ls
ID                            HOSTNAME                STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
lczsoz8fc2zptu6lugj6ouycr *   production-manager      Ready     Active         Leader           25.0.16
n6b9p4o4wdttm4xlwxd07ujtd     production-worker-one   Ready     Active                          25.0.16
6mp83o5pmtk4nmuj7499pxzuu     production-worker-two   Ready     Active                          25.0.16
[ec2-user@production-manager ~]$
```

**Figure A9.2.** *Production Docker swarm*

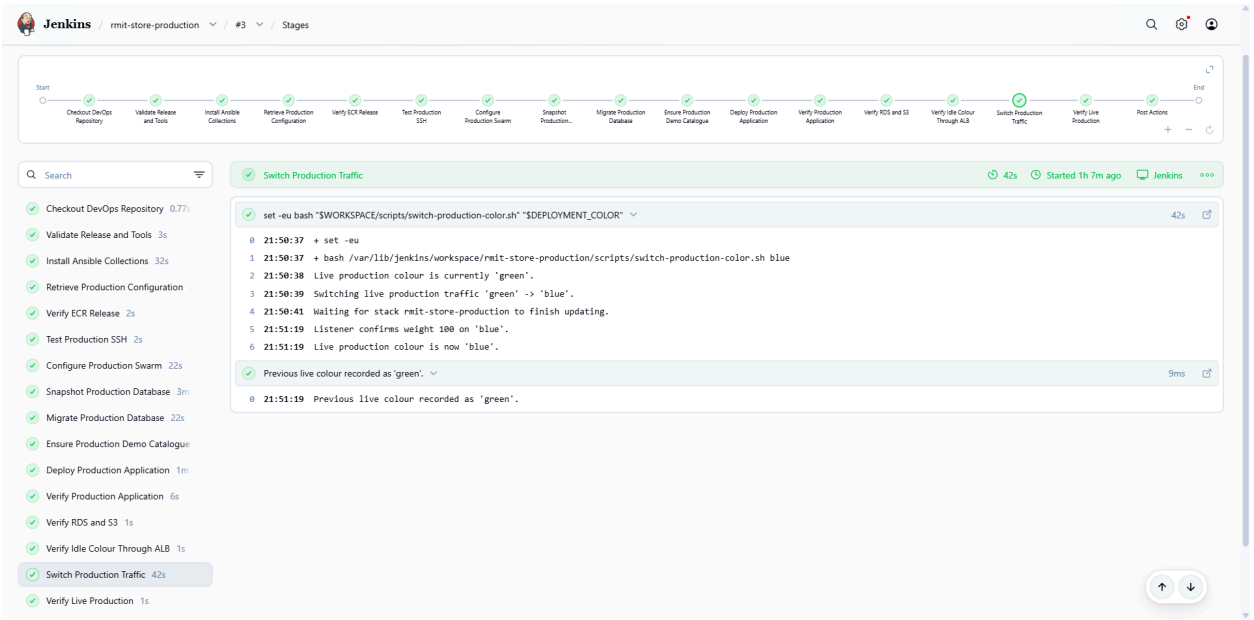



**Figure A9.3.** *Traffic switching*

# Appendix B: Evidence For Advanced Requirements

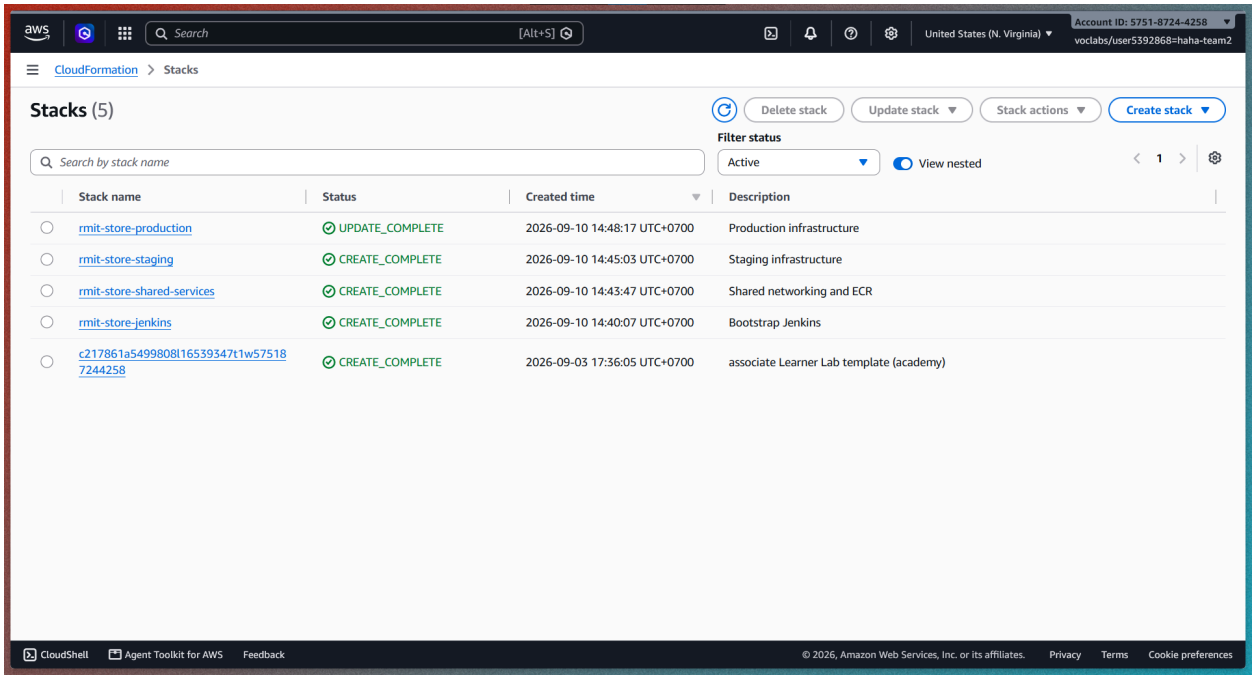



**Figure B1.** *CloudFormation console showing the completed stack statuses*

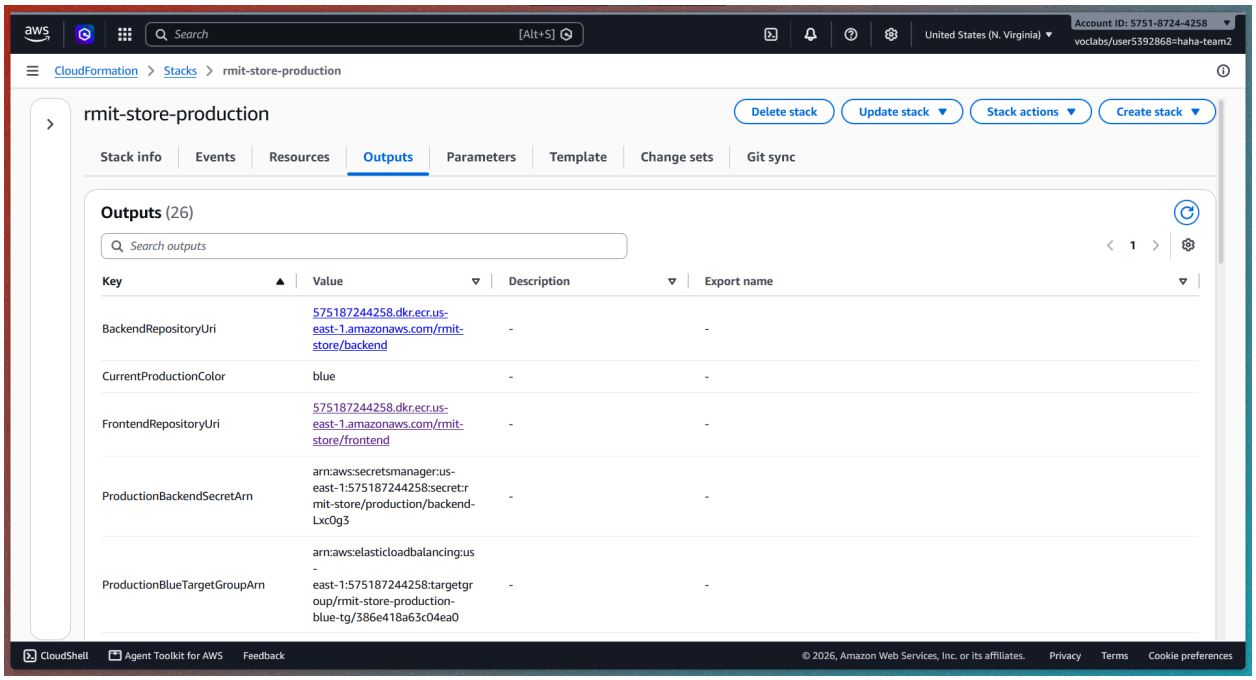



**Figure B2**: *CloudFormation outputs showing production URL, target groups, and current production colour*

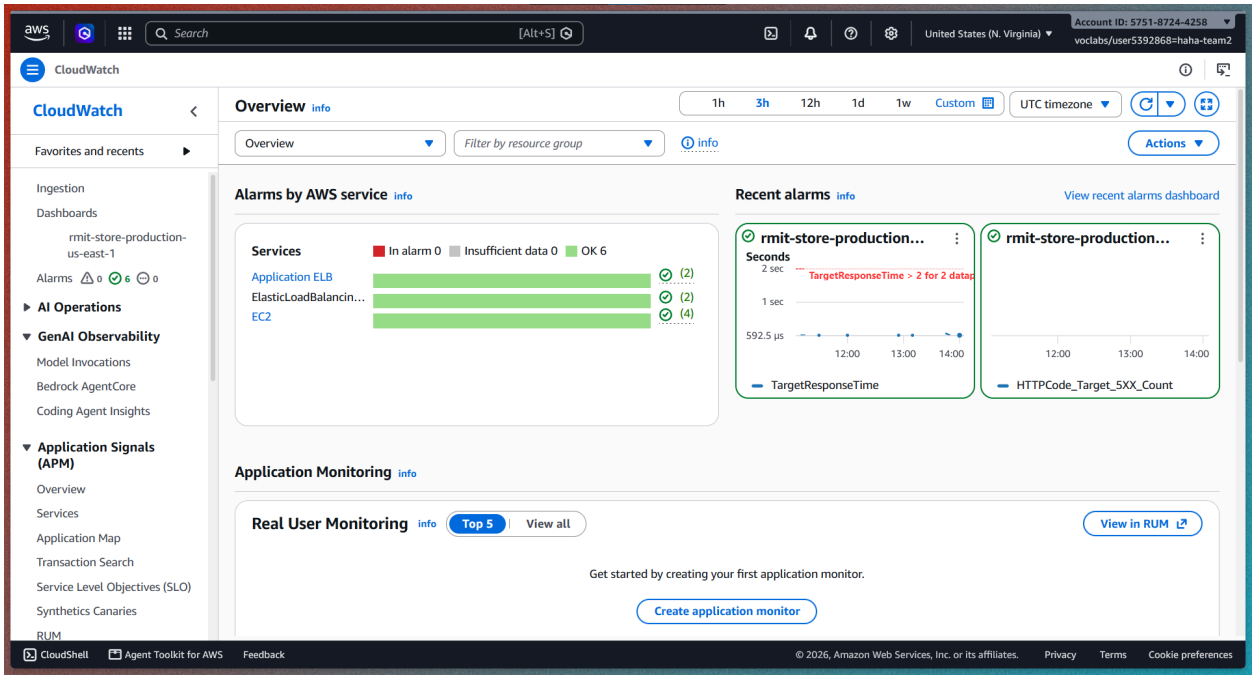



**Figure B3:** *CloudWatch dashboard showing target errors, latency, request count, and CPU utilisation*

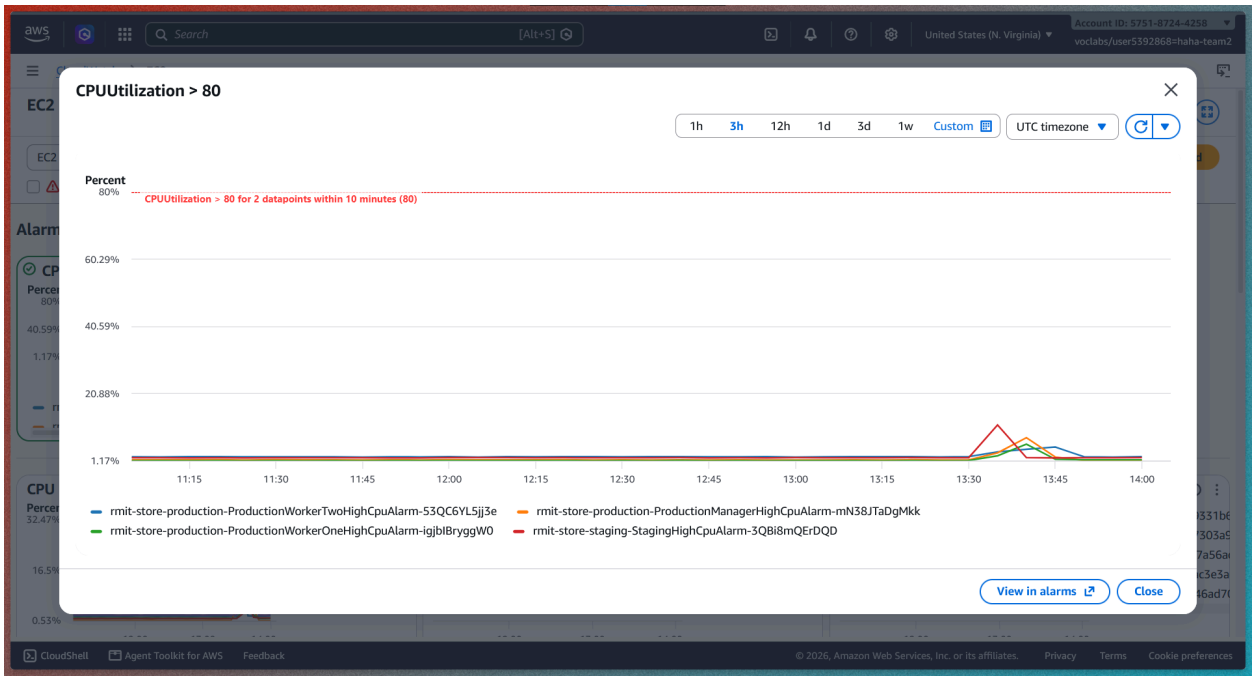



**Figure B4:** *CloudWatch alarm configuration showing the 80% CPU threshold and evaluation periods*

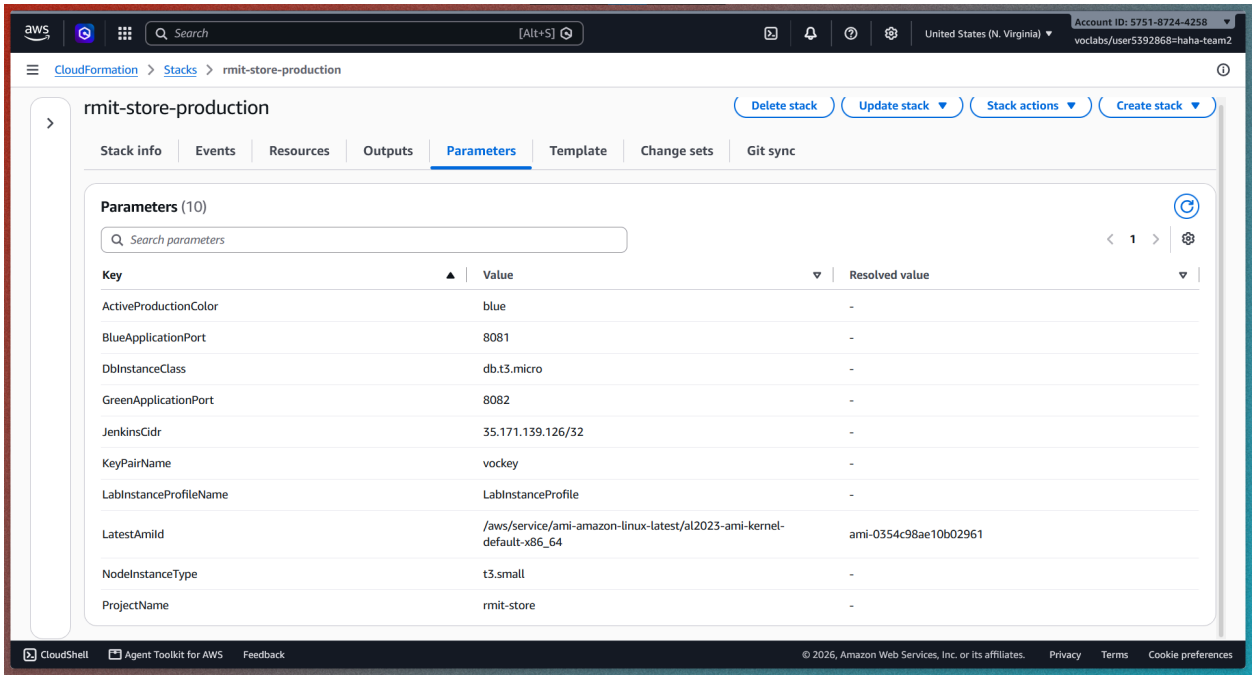



**Figure B5:** *CloudFormations parameters showing different ports for different deployment colors*

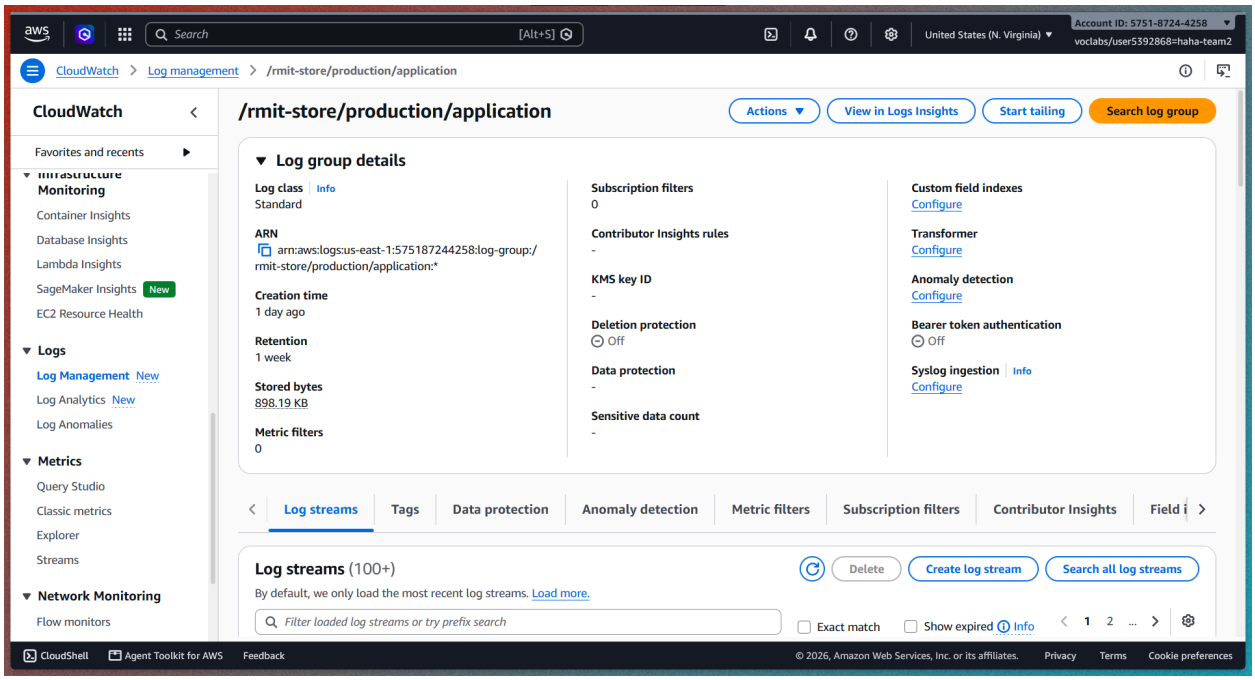



**Figure B6:** *CloudWatch log group showing /rmit-store/production/application*

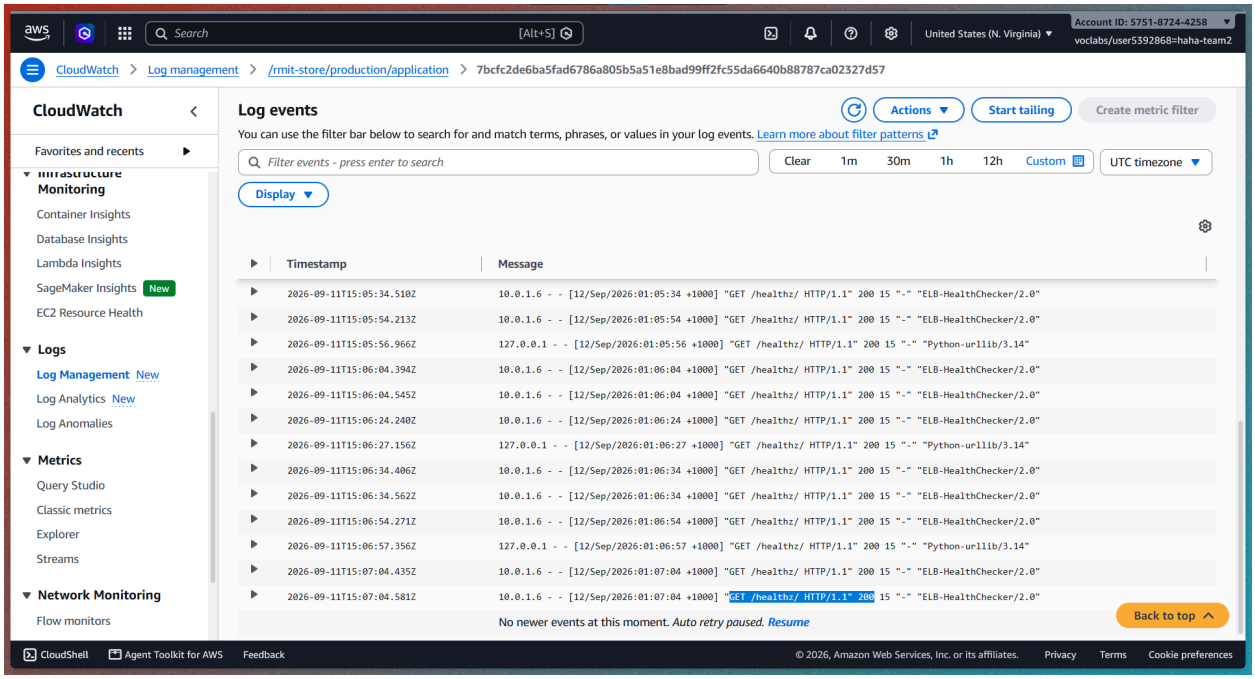



**Figure B7:** *Recent application log stream showing GET /healthz/ HTTP/1.1" 200*

# Appendix C: Evidence for Section 6

Captured on 8 September 2026 against AWS account `749230318571` in us-east-1, running the integrated branch that is being submitted. Every figure refers to build 12 of the `rmit-store-production` job, which deployed release `065d6c036e2b596b6b249a6cb63d833798f531cb` to the green colour, and each is cited from section 5 by number. The public endpoint is `http://rmit-store-production-alb-325185269.us-east-1.elb.amazonaws.com`.

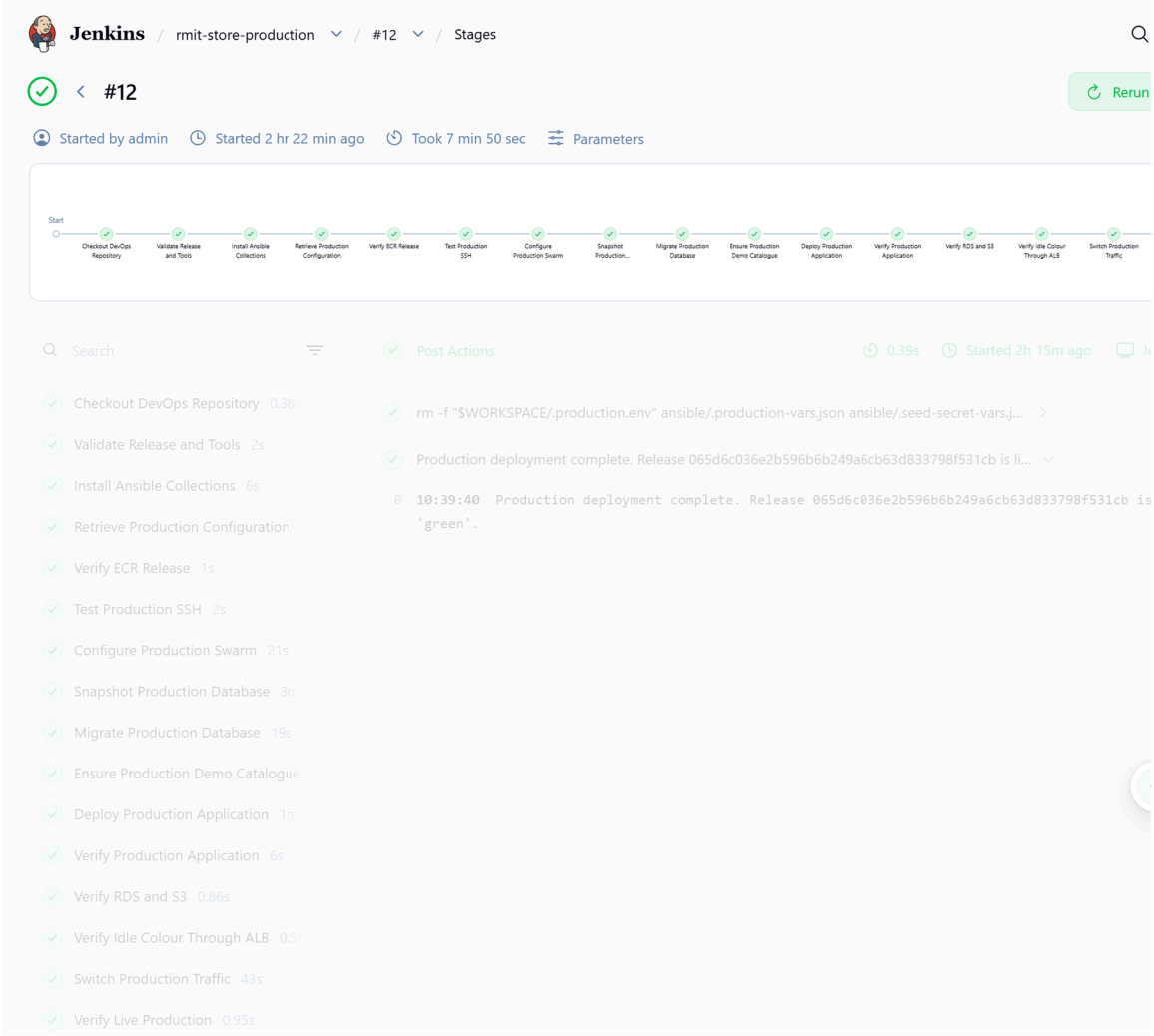



**Figure C1.** *Build 12 of the production pipeline: sixteen stages green in 7 min 50 s, ending "Release 065d6c03 is live on 'green'". Per-stage timings are visible, including the database snapshot at 3 min and the traffic switch at 43s.*

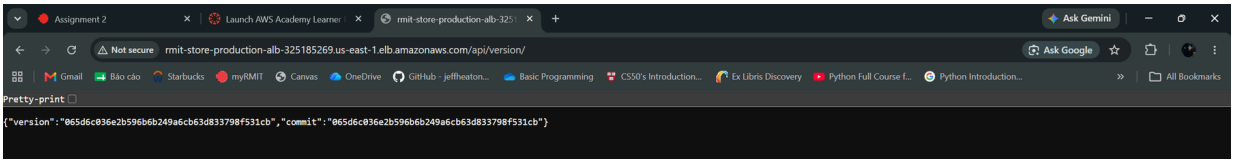



**Figure C2.** *The public endpoint reporting release 065d6c03 after build 12. This ties every screenshot that follows to one specific pipeline run.*

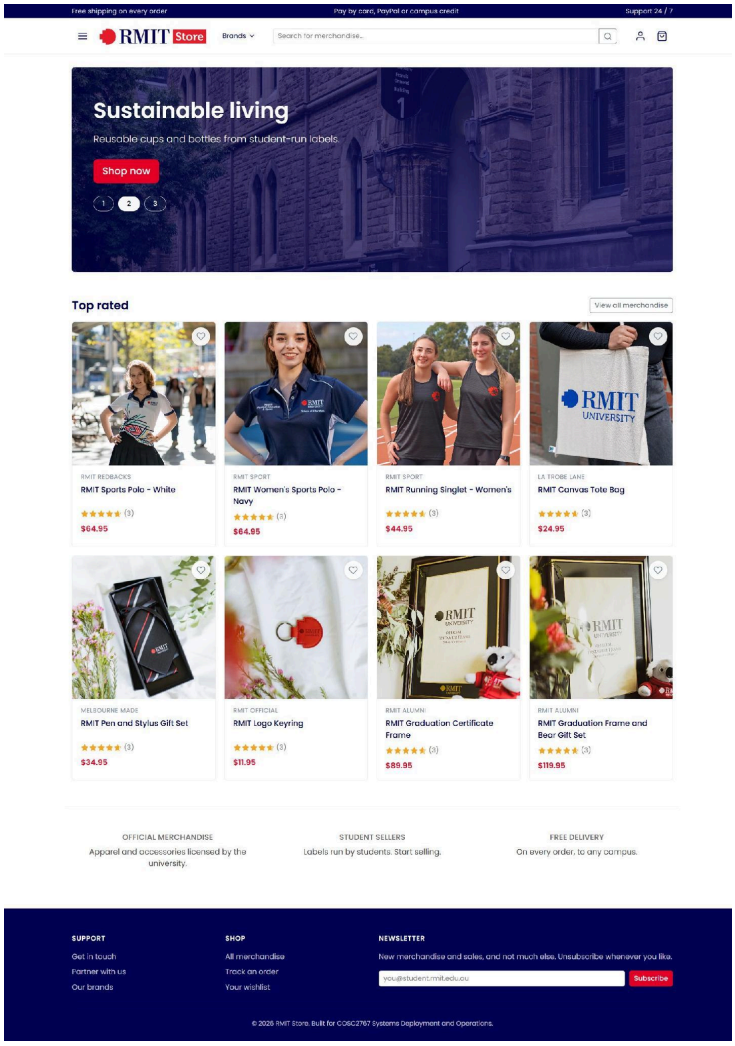



**Figure C3.** *The storefront served through the production ALB, with product imagery loaded from the shared S3 bucket by presigned URL.*

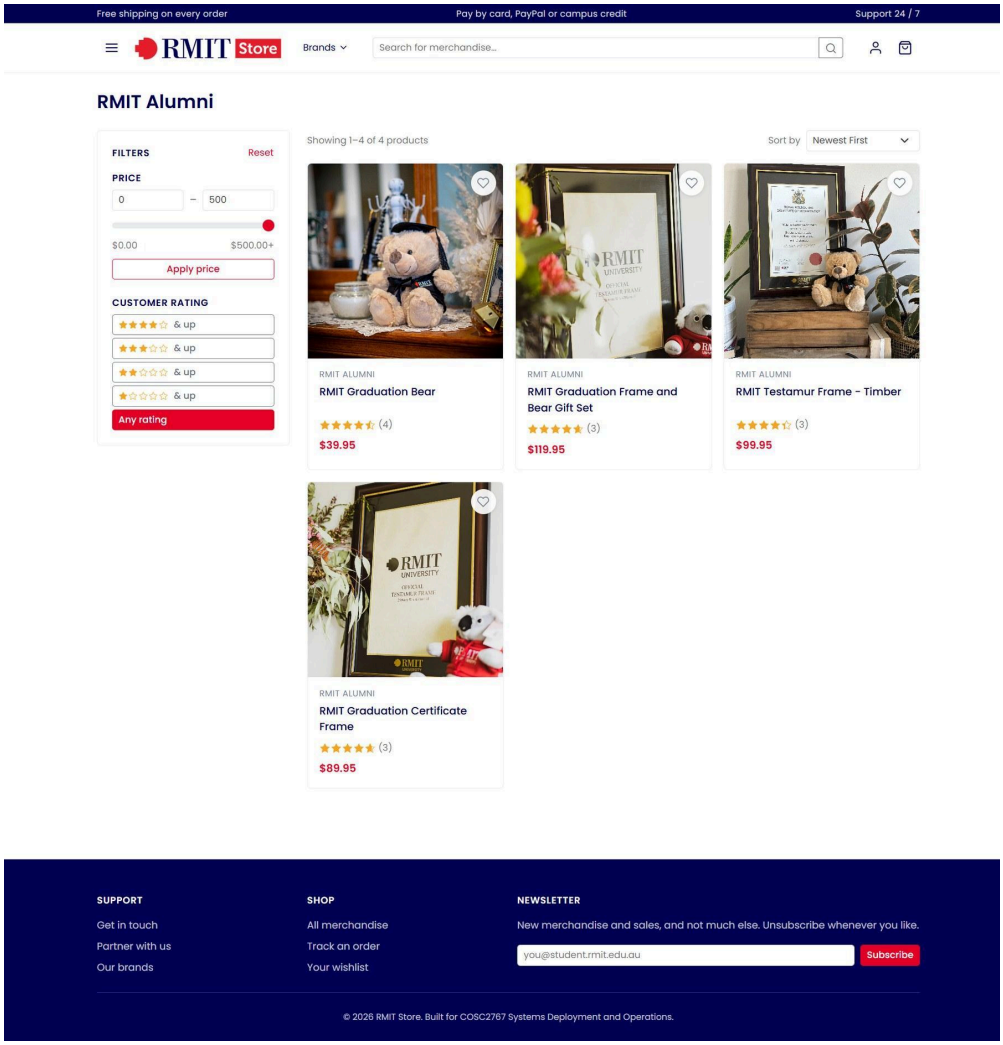



**Figure C4.** *Brand listing with working price and rating filters, four of four products rendering their images.*

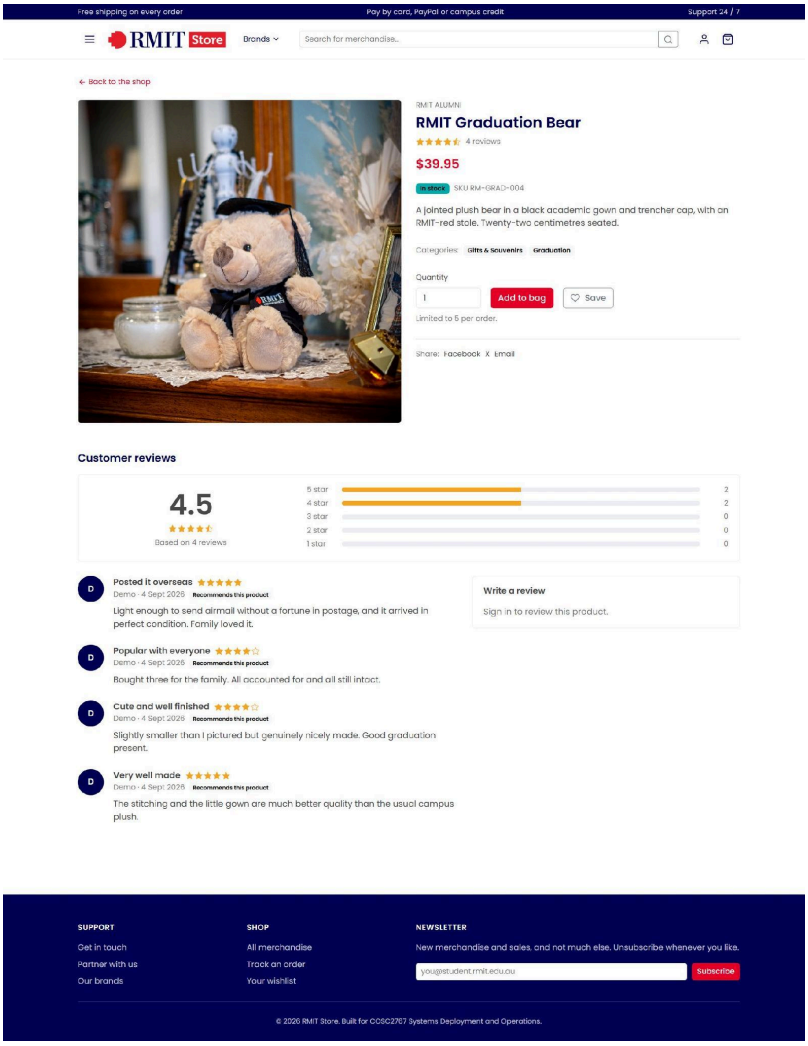



**Figure C5.** *Product detail with price, SKU, stock and four seeded customer reviews. The seeding stage produced usable data, not merely a task that exited zero.*

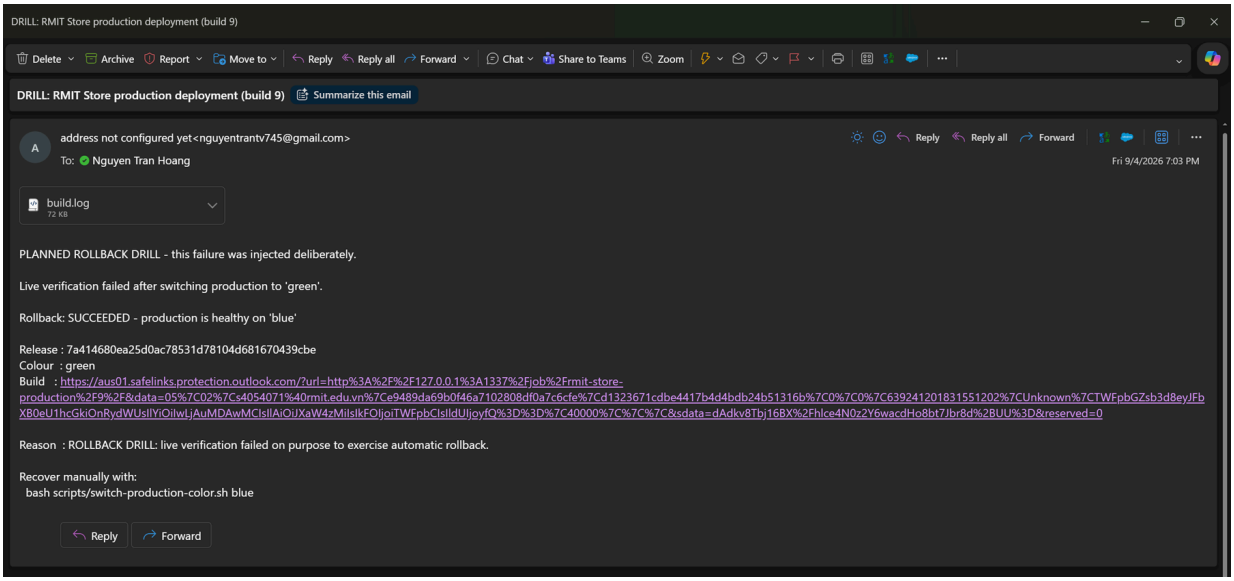



**Figure C6.** *The alert from the rollback drill in build 9. The subject is labelled DRILL, the body opens "PLANNED ROLLBACK DRILL", and it reports "Rollback: SUCCEEDED - production is healthy on 'blue'". Automatic recovery and failure alerting are evidenced together.*

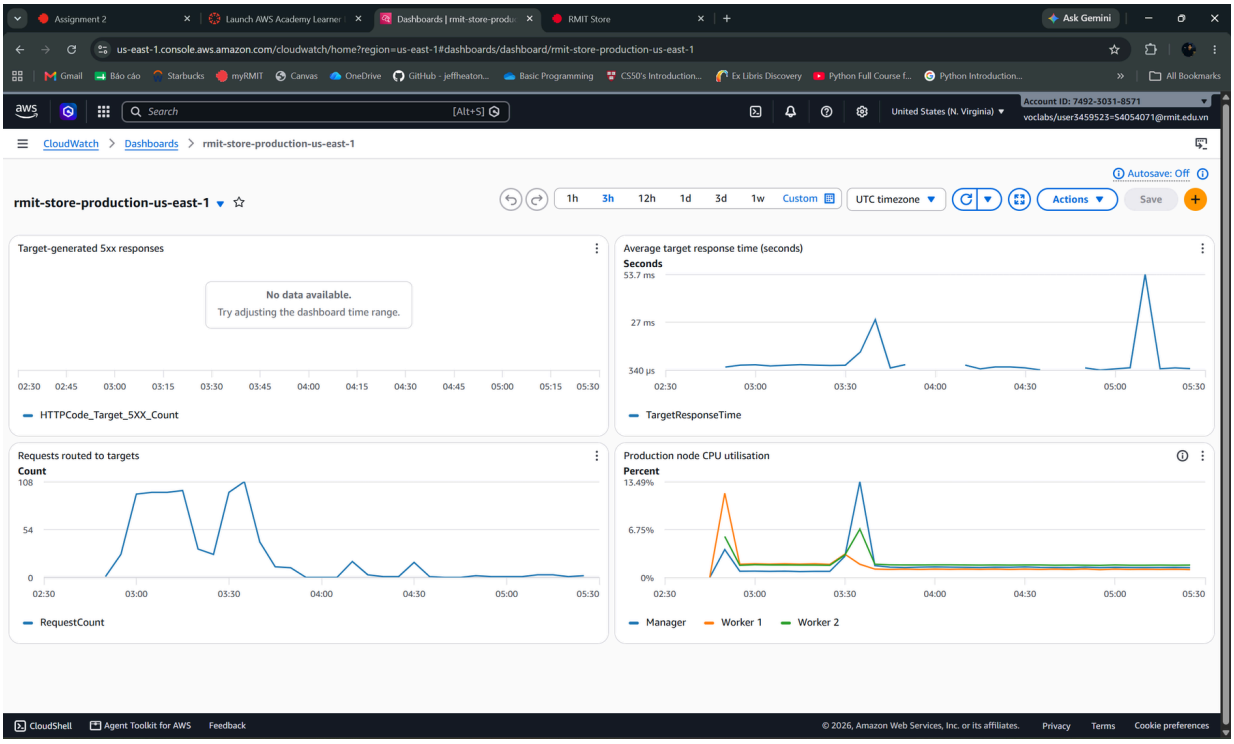



**Figure C7.** *The production CloudWatch dashboard. The target 5xx widget reads "No data available" across the whole window, an independent corroboration of the curl sampling; the two CPU spikes are the two deployments.*

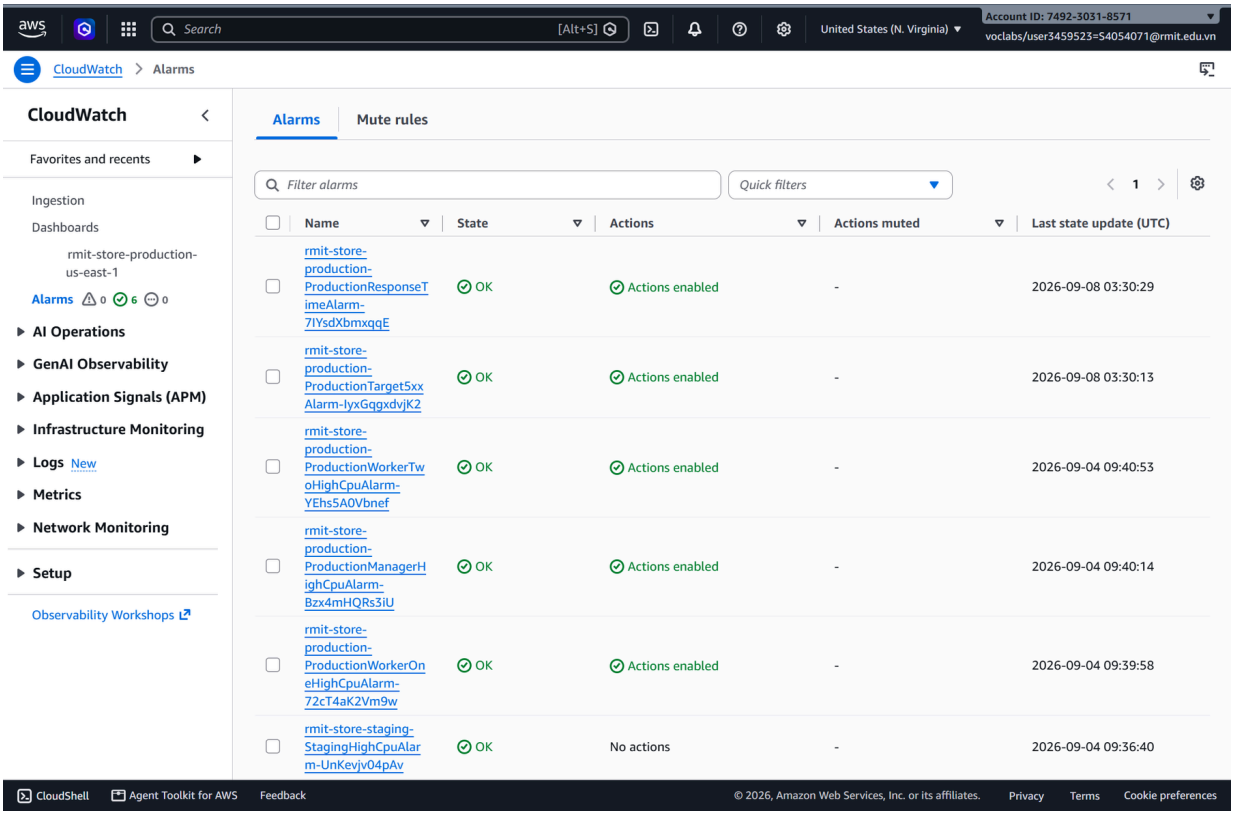



**Figure C8.** *Six alarms, all OK. The five production alarms show "Actions enabled" and publish to SNS; the staging alarm shows "No actions", so alerting was attached deliberately rather than switched on everywhere.*

# Appendix D: Tools & Technologies Table

| Tools & Technology | Purpose/Role |
|---|---|
| Github | Stores application source, tests and deployment code in the assigned private repository. Supports collaboration, version control and webhook-triggered builds. |
| Jenkins | Coordinates the pipeline through application, infrastructure, production and database-restore jobs. Records build results and deployment logs. |
| Docker and Docker Swarm | Package application components into containers and manage deployed services. The Vue frontend is built into static files served by nginx, while Django runs through gunicorn. |
| ECR | Stores versioned backend and frontend container images for deployment. |
| CloudFormation | Defines and provisions the Jenkins bootstrap, shared services, staging and production infrastructure through version-controlled templates. |
| Ansible | Configures deployment hosts and automates application deployment tasks. |
| EC2 & Application Load Balancer | Provide application compute capacity and route requests to healthy deployment targets. |
| PostgreSQL & RDS | Provide relational database storage. Staging uses containerised PostgreSQL, while production uses Amazon RDS. |
| S3 | Stores product images outside individual application containers, allowing replicas to access shared media. |
| Secrets Manager, SSM Parameter Store, Jenkins credentials | Manage sensitive configuration and credentials without committing their values to the repository. |
| Pytest, Vitest, Playwright | Run backend, frontend and browser-based tests at appropriate points in the pipeline. |
| ShellCheck & cfn-lint | Check shell scripts and CloudFormation templates for static-analysis findings before deployment proceeds. |
| CloudWatch & SNS | Provide monitoring, alarms and email notifications about operational conditions and deployment outcomes. |

| | |
|---|---|
| Bash scripts & Custom setup wizard | Provide a common entry point for setup, pipeline status, credential retrieval, Jenkins access, reset and database-restore rehearsal. |

**Table A.** *Tools & Technologies*

# Appendix E: Contribution table

| Member | Student ID | Contribution (%) | Task(s) |
|---|---|---|---|
| Ma Thanh Long Mason | S4042629 | 20% | **Team Management:**<br>- Coordinating meetings<br>- Managing Github repository<br>**Pipeline:**<br>- Designing pipeline<br>- Configuring AWS Services<br>- Creating custom wizard<br>**Report:**<br>- Section 1<br>- Section 5<br>- Formatting<br>**Presentation:**<br>- Problem statement<br>- Infrastructure overview<br>- Quick setup guide<br>- Video Editing |
| Do Hao Nhien | S3994477 | 20% | **Pipeline:**<br>- Production deployment and operation (jenkins/Jenkinsfile.production)<br>- Configuring the three-node Docker Swarm with Ansible<br>- Deploying the frontend and backend replicas from ECR<br>- Integrating production with RDS and S3<br>- Handling secrets and migrations<br>- Adding safe fresh-database seeding<br>**Report:**<br>- Section 4<br>- Formatting<br>**Presentation:**<br>- Production architecture / operations<br>- Data protection with Secrets<br>- Show and explain completed restore-rehearsal report<br>- Video Editing |
| Tran Hoang Nguyen | S4054071 | 20% | **Pipeline:**<br>- Production blue-green deployment (jenkins/Jenkinsfile.production)<br>- Idle-colour verification and live verification through the ALB<br>- Traffic cutover through the |

| | | | |
|---|---|---|---|
| | | | ActiveProductionColor stack parameter<br>- Automatic rollback and the ROLLBACK_DRILL rehearsal<br>- SNS alerting for deployment, rollback and drill outcomes<br>- Production database seeding playbook<br>- Fixes: Docker SDK interpreter, seed credential delivery, rolling-update wait gate<br>**Report:**<br>- Section 6<br>- Appendix C<br>- References<br>- Conclusion<br>**Presentation:**<br>- Blue-green cutover, rollback and alerting segment<br>- Endpoint sampling evidence for the cutover and the rollback drill |
| Bui Linh Nhi | S4086154 | 20% | **Pipeline:**<br>- Designing staging processes<br>**Report:**<br>- Section 2<br>**Presentation:**<br>- Staging pipeline |
| Nguyen Tran Nhat Huy | S3978239 | 20% | **Pipeline:**<br>- Test design and integration into Jenkinsfile.application pipeline for backend, frontend, and end-to-end.<br>**Report:**<br>- Section 3<br>**Presentation:**<br>- Tests and quality gates<br>- Database Recovery |

**Table F.** *Contribution table (full)*